

Towards fast and side-channel resistant implementations of Hamming Quasi-Cyclic

A case study on ARM Cortex-M4 devices

Domenico Cacace

May 4, 2023

Overview

1. Introduction
2. Background
   - Polynomial Multplication
   - Coding theory
   - Hamming Quasi-Cyclic
   - Side channel attacks
3. Implementation
   - Polynomial multiplication
   - Masking
4. Experimental results
   - Performance
   - Side-channel resistance
   - Comparison with other cryptosystems
5. Conclusions

Introduction

Kerchoff's Principle

Everything about a cryptosystem is publicly known, **except for the key**

Symmetric encryption

Key: a n-bit string

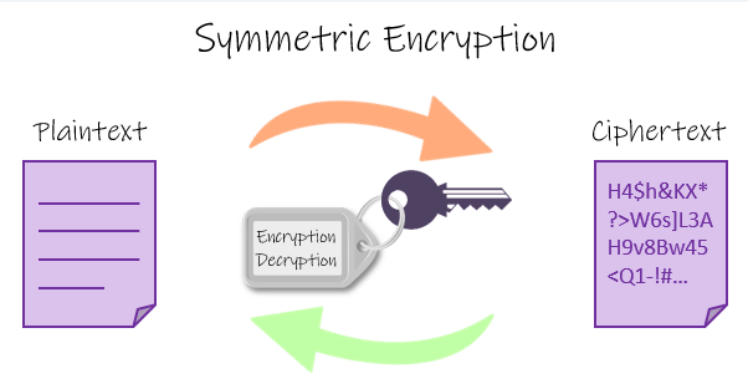
Asymmetric encryption

Key: the product of two prime numbers, *i.e.* $n = p \cdot q$

Kerchoff's Principle

Everything about a cryptosystem is publicly known, **except for the key**

Symmetric encryption



Key: a n-bit string

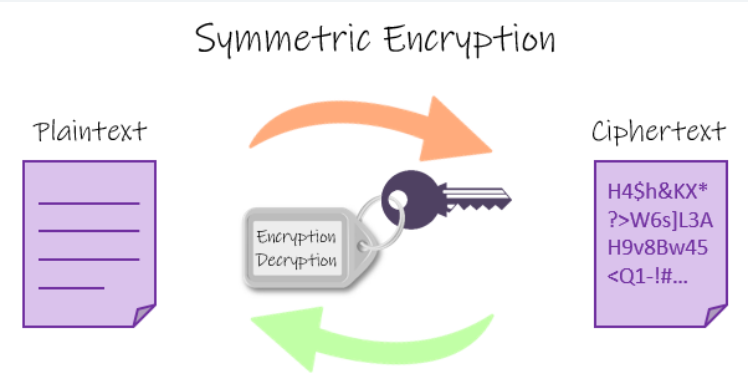
Asymmetric encryption

Key: the product of two prime numbers, *i.e.* $n = p \cdot q$

Kerchoff's Principle

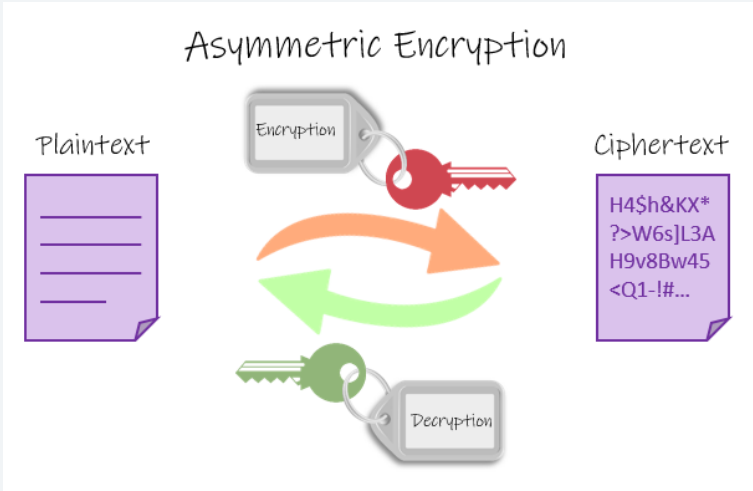
Everything about a cryptosystem is publicly known, **except for the key**

Symmetric encryption



Key: a n-bit string

Asymmetric encryption



Key: the product of two prime numbers, *i.e.* $n = p \cdot q$

Cryptanalytic attacks

What are the best attacks on modern cryptosystems?

On Classical Computers

- Symmetric: exhaustive search, $\mathcal{O}(2^n)$
- Asymmetric: General Number Field Sieve, $\approx \mathcal{O}(2^{n-\epsilon})$

On Quantum Computers

- Symmetric: Grover's algorithm, $\mathcal{O}(\sqrt{2^n})$
- Asymmetric: Shor's algorithm, $\approx \mathcal{O}((\log n)^2)$

Shor's algorithm can break asymmetric cryptosystems in polynomial time!

Cryptanalytic attacks

What are the best attacks on modern cryptosystems?

On Classical Computers

- Symmetric: exhaustive search, $\mathcal{O}(2^n)$
- Asymmetric: General Number Field Sieve, $\approx \mathcal{O}(2^{n-\epsilon})$

On Quantum Computers

- Symmetric: Grover's algorithm, $\mathcal{O}(\sqrt{2^n})$
- Asymmetric: Shor's algorithm, $\approx \mathcal{O}((\log n)^2)$

Shor's algorithm can break asymmetric cryptosystems in polynomial time!

Cryptanalytic attacks

What are the best attacks on modern cryptosystems?

On Classical Computers

- Symmetric: exhaustive search, $\mathcal{O}(2^n)$
- Asymmetric: General Number Field Sieve, $\approx \mathcal{O}(2^{n-\epsilon})$

On Quantum Computers

- Symmetric: Grover's algorithm, $\mathcal{O}(\sqrt{2^n})$
- Asymmetric: Shor's algorithm, $\approx \mathcal{O}((\log n)^2)$

Shor's algorithm can break asymmetric cryptosystems in polynomial time!

Cryptanalytic attacks

What are the best attacks on modern cryptosystems?

On Classical Computers

- Symmetric: exhaustive search, $\mathcal{O}(2^n)$
- Asymmetric: General Number Field Sieve, $\approx \mathcal{O}(2^{n-\epsilon})$

On Quantum Computers

- Symmetric: Grover's algorithm, $\mathcal{O}(\sqrt{2^n})$
- Asymmetric: Shor's algorithm, $\approx \mathcal{O}((\log n)^2)$

Shor's algorithm can break asymmetric cryptosystems in polynomial time!

Cryptanalytic attacks

What are the best attacks on modern cryptosystems?

On Classical Computers

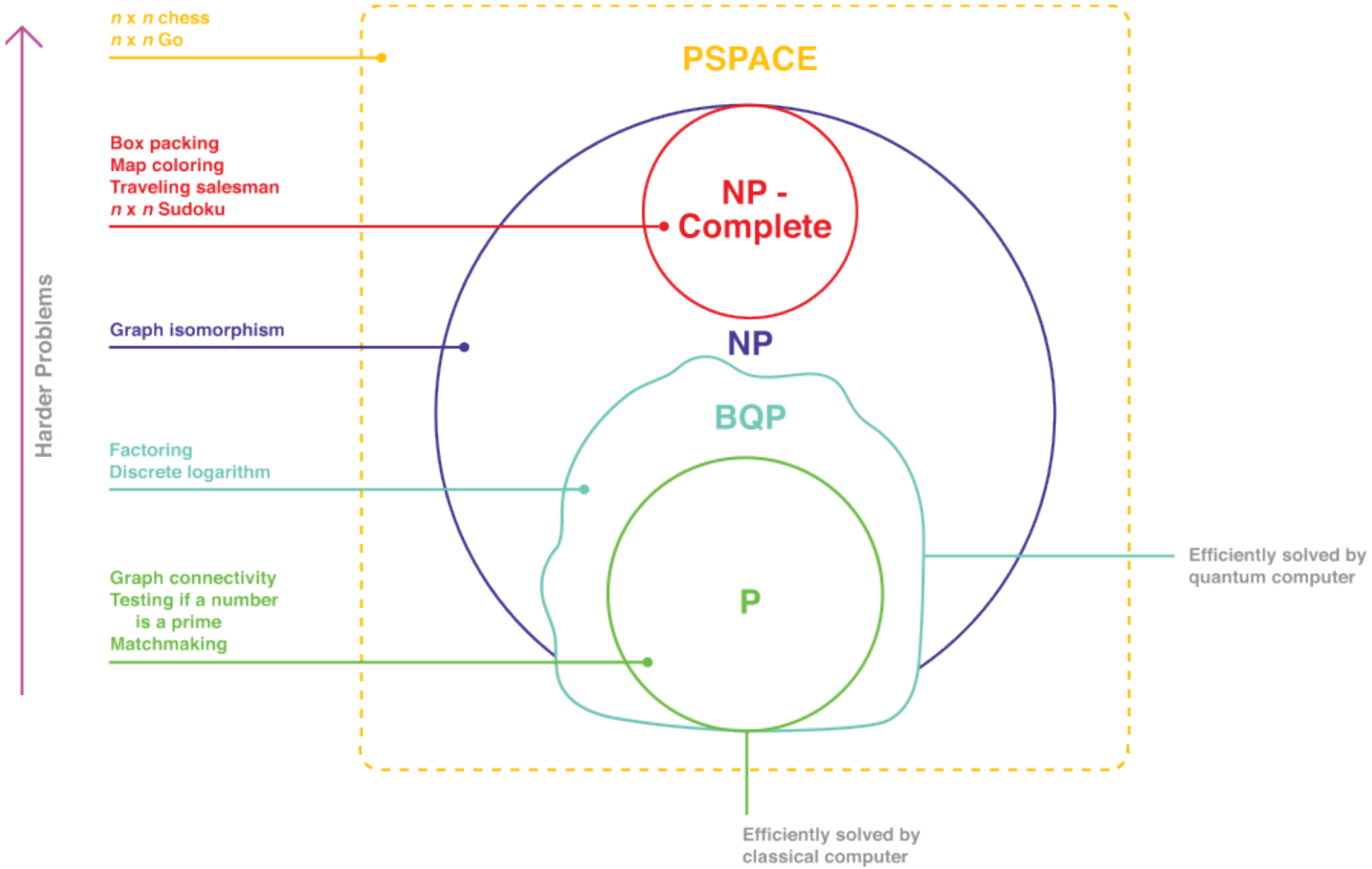
- Symmetric: exhaustive search, $\mathcal{O}(2^n)$
- Asymmetric: General Number Field Sieve, $\approx \mathcal{O}(2^{n-\epsilon})$

On Quantum Computers

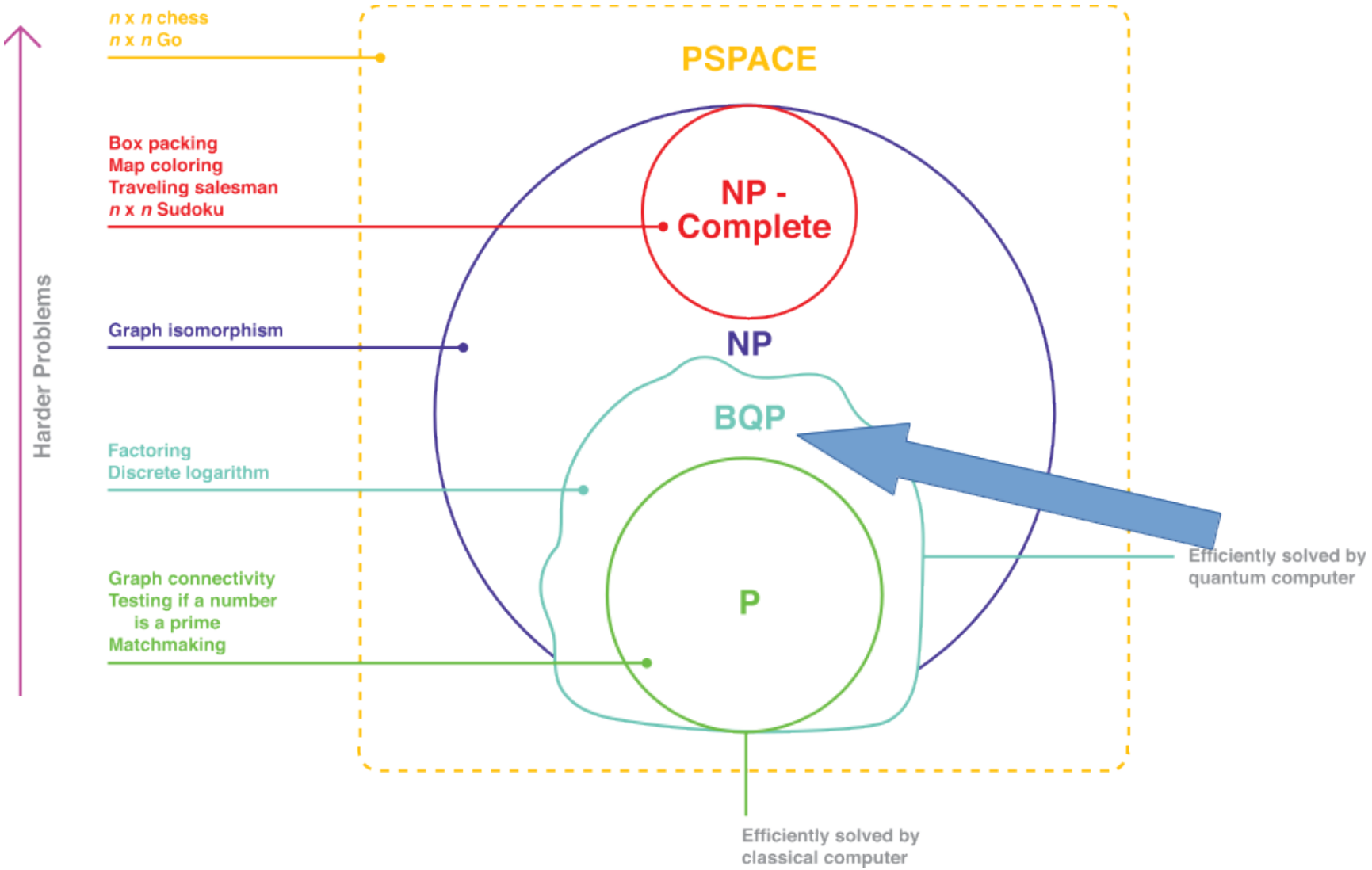
- Symmetric: Grover's algorithm, $\mathcal{O}(\sqrt{2^n})$
- Asymmetric: Shor's algorithm, $\approx \mathcal{O}((\log n)^2)$

Shor's algorithm can break asymmetric cryptosystems in polynomial time!

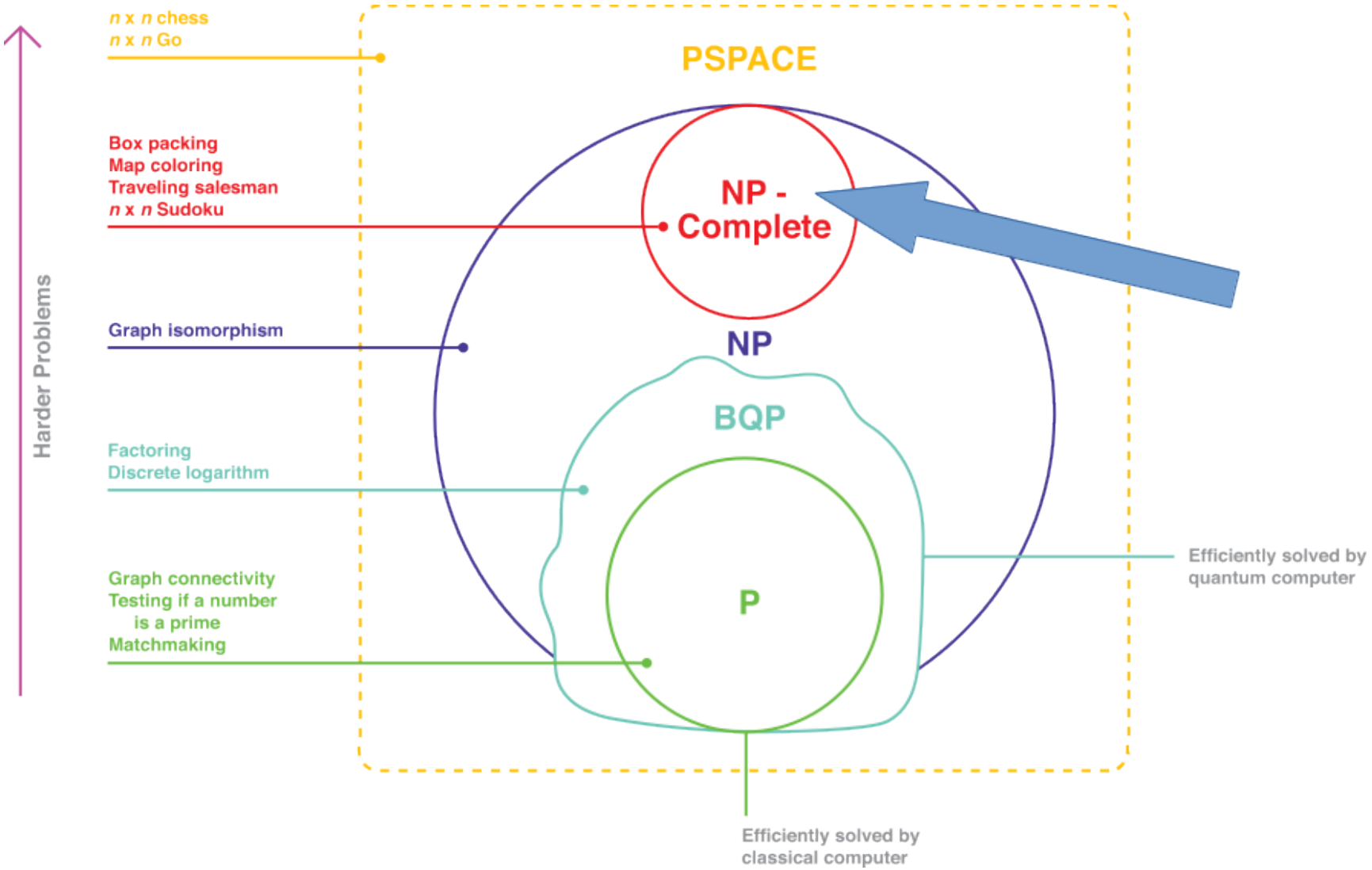
The Computational Complexity Zoo



The Computational Complexity Zoo



The Computational Complexity Zoo



NIST Post-Quantum Cryptography Standardization

Timeline

- 26/02/16: Announcement of NIST's call for submissions
- 21/12/17: Announcement of the first round candidates (69)
- 30/01/19: Announcement of the second round candidates (26)
- 22/07/20: Announcement of the third round candidates (7 finalists, 8 alternatives)
- 05/07/22: Announcement of the fourth round candidates (4)
- 2022-2024: Publication of the standard drafts

NIST Post-Quantum Cryptography Standardization

Timeline

- 26/02/16: Announcement of NIST's call for submissions
- 21/12/17: Announcement of the first round candidates (69)
- 30/01/19: Announcement of the second round candidates (26)
- 22/07/20: Announcement of the third round candidates (7 finalists, 8 alternatives)
- 05/07/22: Announcement of the fourth round candidates (4)
- 2022-2024: Publication of the standard drafts

NIST Post-Quantum Cryptography Standardization

Timeline

- 26/02/16: Announcement of NIST's call for submissions
- 21/12/17: Announcement of the first round candidates (69)
- 30/01/19: Announcement of the second round candidates (26)
- 22/07/20: Announcement of the third round candidates (7 finalists, 8 alternatives)
- 05/07/22: Announcement of the fourth round candidates (4)
- 2022-2024: Publication of the standard drafts

Standardization candidates

Third Round winner

- CRYSTALS-Kyber: lattice-based

NIST decided to standardize at least another PQC scheme, in case lattice-based crypto fails

Fourth Round candidates

- ~~**S**upersingular **I**sogeny **K**ey **E**ncapsulation~~
- **B**it Fl**i**pping **K**ey **E**ncapsulation [ABB+22]
- **H**amming **Q**uasi-**C**yclic [MAB+18]
- Classic McEliece [BCC+22]

BIKE, Classic McEliece and HQC are all code-based cryptosystems.

Standardization candidates

Third Round winner

- CRYSTALS-Kyber: lattice-based

NIST decided to standardize at least another PQC scheme, in case lattice-based crypto fails

Fourth Round candidates

- ~~**S**upersingular **I**sogeny **K**ey **E**ncapsulation~~
- **B**it Fl**i**pping **K**ey **E**ncapsulation [ABB+22]
- **H**amming **Q**uasi-**C**yclic [MAB+18]
- Classic McEliece [BCC+22]

BIKE, Classic McEliece and HQC are all code-based cryptosystems.

Standardization candidates

Third Round winner

- CRYSTALS-Kyber: lattice-based

NIST decided to standardize at least another PQC scheme, in case lattice-based crypto fails

Fourth Round candidates

- ~~**S**upersingular **I**sogeny **K**ey **E**ncapsulation~~
- **B**it Fl**i**pping **K**ey **E**ncapsulation [ABB+22]
- **H**amming **Q**uasi-**C**yclic [MAB+18]
- Classic McEliece [BCC+22]

BIKE, Classic McEliece and HQC are all code-based cryptosystems.

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Overview

The target of this thesis is Hamming Quasi-Cyclic

Goals

- High performance implementation for Cortex-M4
- Protection against timing attacks [GHJ+22, WBB+22]
- Protection against power attacks [SRSW20, SHR+22]

Roadmap

- Evaluate various polynomial multiplication algorithms
- Assess the constant-time behavior of all components
- Implement a masking scheme for the PKE primitives
- Compare the performance with the other Round 4 candidates

Background

Polynomial multiplication 101

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$
$$b(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n$$

Our goal: calculate:

$$c(x) = a(x) \cdot b(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{2n} x^{2n}$$

- Schoolbook multiplication
- Karatsuba algorithm
- FFT-based approach

Polynomial multiplication 101

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$
$$b(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n$$

Our goal: calculate:

$$c(x) = a(x) \cdot b(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{2n} x^{2n}$$

- Schoolbook multiplication
- Karatsuba algorithm
- FFT-based approach

Polynomial multiplication 101

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$
$$b(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n$$

Our goal: calculate:

$$c(x) = a(x) \cdot b(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{2n} x^{2n}$$

- Schoolbook multiplication
- Karatsuba algorithm
- FFT-based approach

Schoolbook multiplication

Multiply each coefficient of $a(x)$ with each coefficient of $b(x)$

$$
\begin{aligned}
c(x) =& a(x) \cdot b(x) \\
=& a_0 \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\
&+ a_1 x \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\
&+ \cdots + \\
&+ a_n x^n \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n)
\end{aligned}
$$

- n^2 multiplications
- n^2 additions

Overall complexity: $\mathcal{O}(n^2)$

Schoolbook multiplication

Multiply each coefficient of $a(x)$ with each coefficient of $b(x)$

$$\begin{aligned} c(x) =& a(x) \cdot b(x) \\ =& a_0 \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ a_1 x \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ \cdots + \\ &+ a_n x^n \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) \end{aligned}$$

- n^2 multiplications
- n^2 additions

Overall complexity: $O(n^2)$

Schoolbook multiplication

Multiply each coefficient of $a(x)$ with each coefficient of $b(x)$

$$\begin{aligned} c(x) =& a(x) \cdot b(x) \\ =& a_0 \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ a_1 x \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ \cdots + \\ &+ a_n x^n \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) \end{aligned}$$

- n^2 multiplications
- n^2 additions

Overall complexity: $\mathcal{O}(n^2)$

Schoolbook multiplication

Multiply each coefficient of $a(x)$ with each coefficient of $b(x)$

$$\begin{aligned} c(x) =& a(x) \cdot b(x) \\ =& a_0 \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ a_1 x \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) + \\ &+ \cdots + \\ &+ a_n x^n \cdot (b_0 + b_1 x + b_2 x^2 + \cdots + b_n x^n) \end{aligned}$$

- n^2 multiplications
- n^2 additions

Overall complexity: $\mathcal{O}(n^2)$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

Karatsuba multiplication

Recursively split $a(x)$ and $b(x)$ into two halves

$$a(x) = a_0(x) + x^m a_1(x) \qquad b(x) = b_0(x) + x^m b_1(x)$$

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

Trick: rewrite $z_1(x)$ as $(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$

Overall complexity: $\mathcal{O}(n^{\log_2 3})$ if $m \approx n/2$

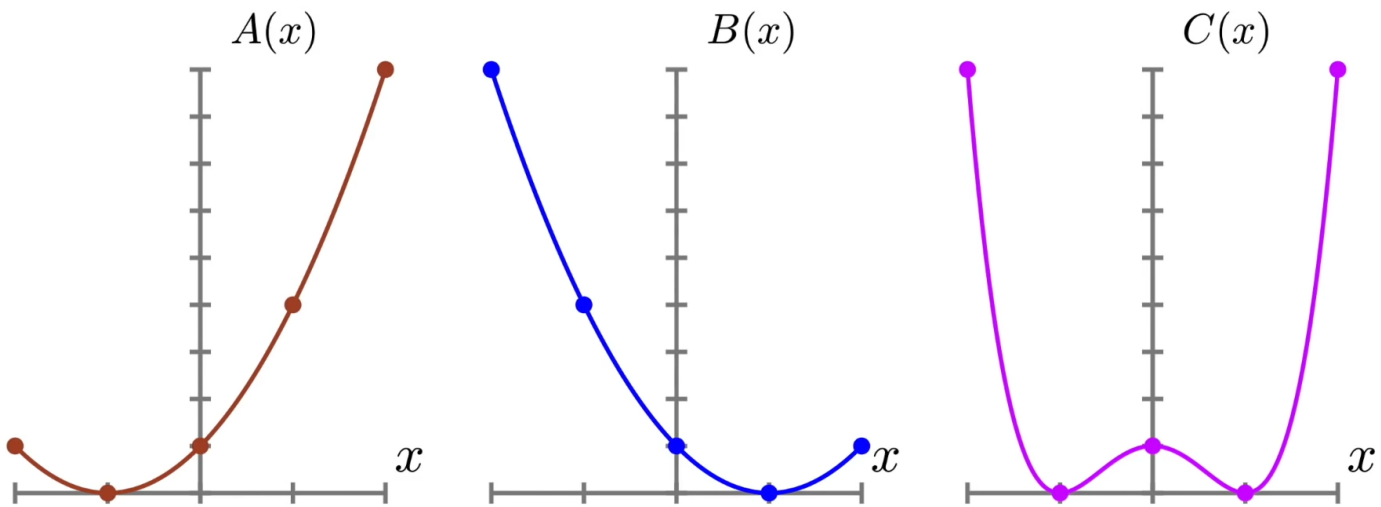
Sample and interpolate

- Sample $a(x)$ and $b(x)$ at $2n+1$ points
- Multiply each pair of samples
- Interpolate the points to get $c(x)$

$$A(x) = x^2 + 2x + 1 \qquad [(-2,1),(-1,0),(0,1),(1,4),(2,9)]$$

$$B(x) = x^2 - 2x + 1 \qquad [(-2,9),(-1,4),(0,1),(1,0),(2,1)]$$

$$C(x) = x^4 - 2x^2 + 1 \qquad [(-2,9),(-1,0),(0,1),(1,0),(2,9)]$$

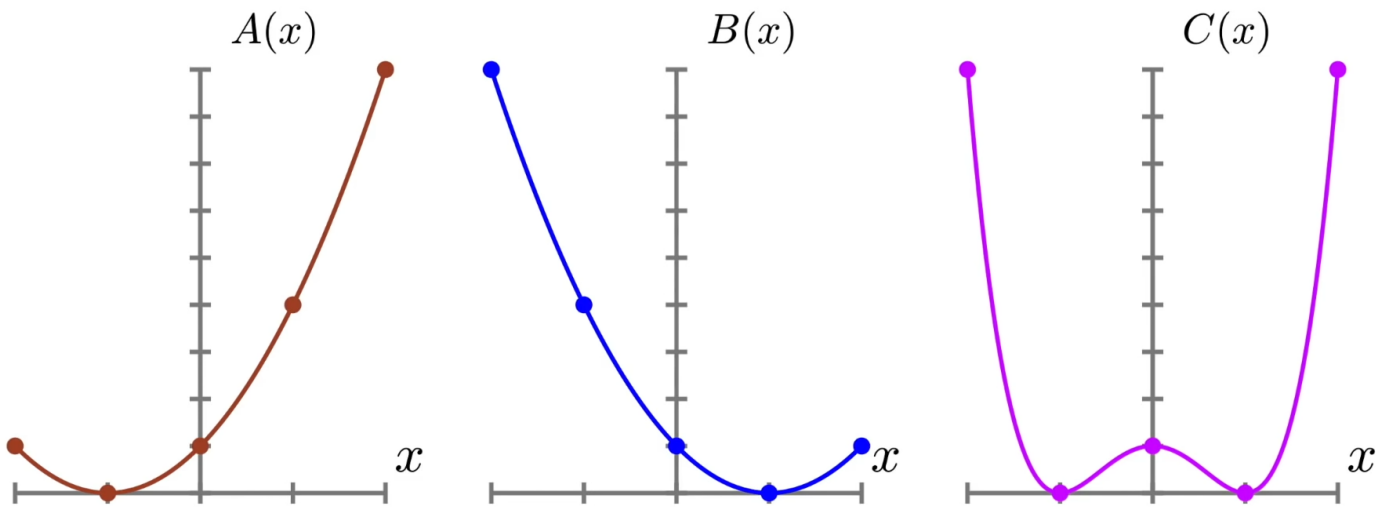


Sample and interpolate

- Sample $a(x)$ and $b(x)$ at $2n+1$ points
- Multiply each pair of samples
- Interpolate the points to get $c(x)$

$$A(x) = x^2 + 2x + 1 \qquad B(x) = x^2 - 2x + 1 \qquad C(x) = x^4 - 2x^2 + 1$$

$$[(-2,1),(-1,0),(0,1),(1,4),(2,9)] \qquad [(-2,9),(-1,4),(0,1),(1,0),(2,1)] \qquad [(-2,9),(-1,0),(0,1),(1,0),(2,9)]$$



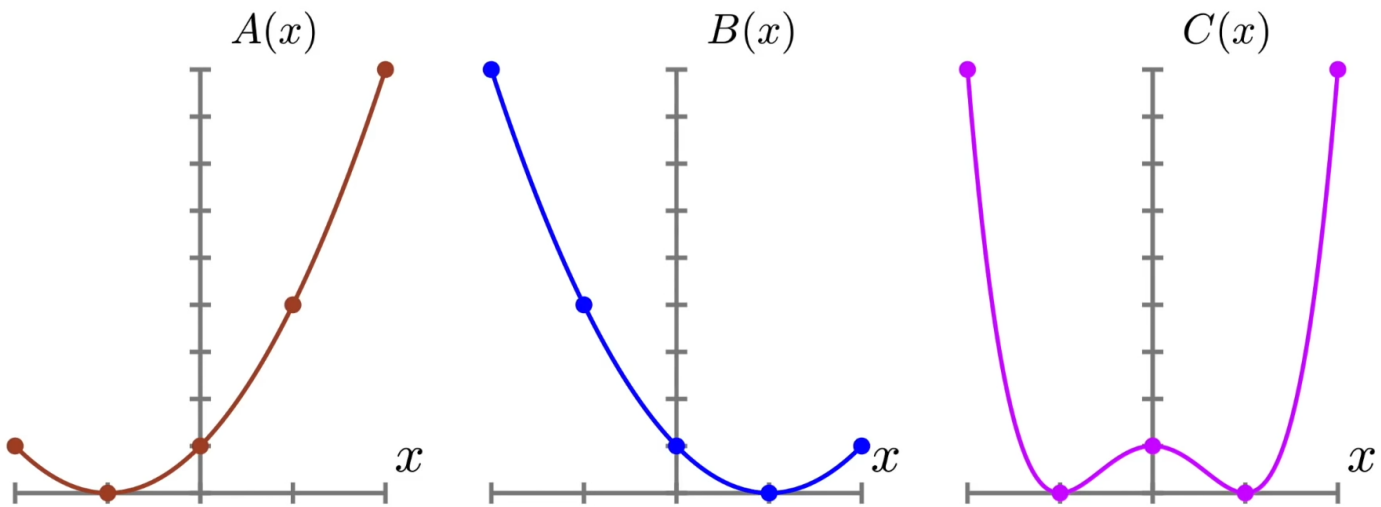
Sample and interpolate

- Sample $a(x)$ and $b(x)$ at $2n+1$ points
- Multiply each pair of samples
- Interpolate the points to get $c(x)$

$$A(x) = x^2 + 2x + 1 \qquad [(-2,1),(-1,0),(0,1),(1,4),(2,9)]$$

$$B(x) = x^2 - 2x + 1 \qquad [(-2,9),(-1,4),(0,1),(1,0),(2,1)]$$

$$C(x) = x^4 - 2x^2 + 1 \qquad [(-2,9),(-1,0),(0,1),(1,0),(2,9)]$$

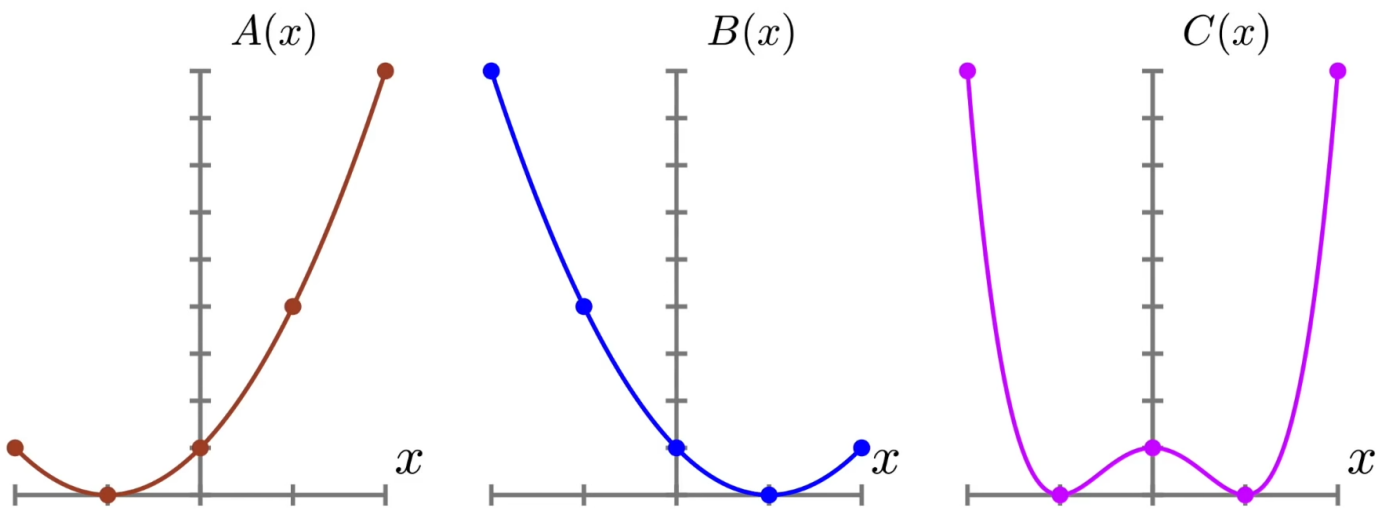


How can this be efficient?!

- Sampling: evaluate the polynomial at $2n+1$ points, $\mathcal{O}(n^2)$
- Pointwise multiplication: $\mathcal{O}(n)$
- Interpolation: up to $\mathcal{O}(n^3)$

$$A(x) = x^2 + 2x + 1 \qquad B(x) = x^2 - 2x + 1 \qquad C(x) = x^4 - 2x^2 + 1$$

$$[(-2,1),(-1,0),(0,1),(1,4),(2,9)] \qquad [(-2,9),(-1,4),(0,1),(1,0),(2,1)] \qquad [(-2,9),(-1,0),(0,1),(1,0),(2,9)]$$

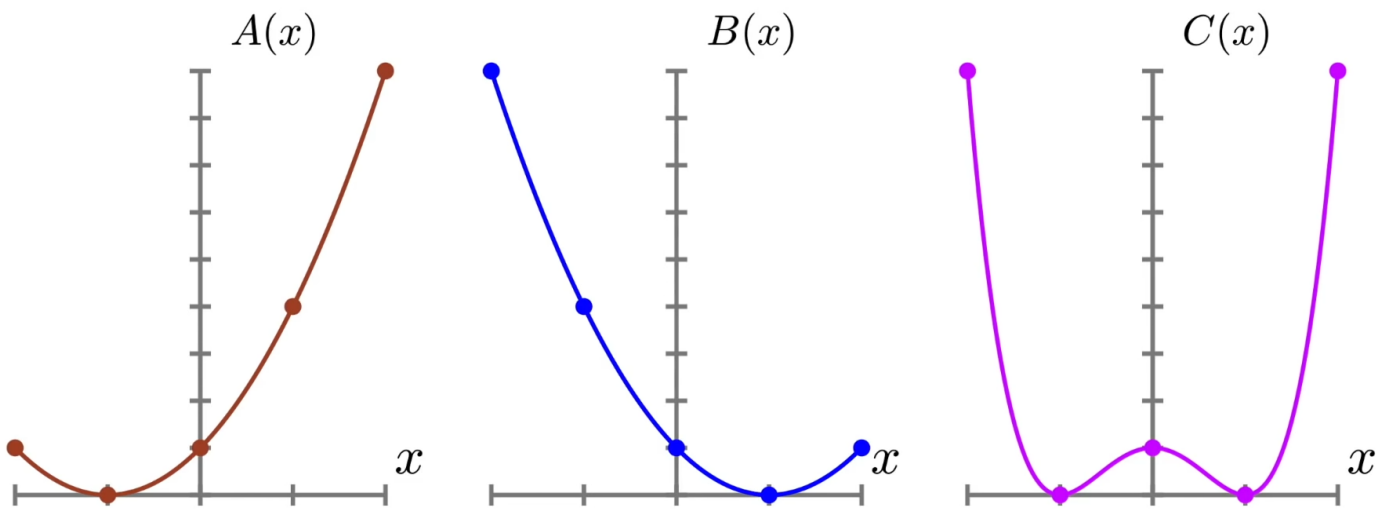


How can this be efficient?!

- Sampling: evaluate the polynomial at $2n+1$ points, $\mathcal{O}(n^2)$
- Pointwise multiplication: $\mathcal{O}(n)$
- Interpolation: up to $\mathcal{O}(n^3)$

$$A(x) = x^2 + 2x + 1 \qquad B(x) = x^2 - 2x + 1 \qquad C(x) = x^4 - 2x^2 + 1$$

$$[(-2,1),(-1,0),(0,1),(1,4),(2,9)] \qquad [(-2,9),(-1,4),(0,1),(1,0),(2,1)] \qquad [(-2,9),(-1,0),(0,1),(1,0),(2,9)]$$



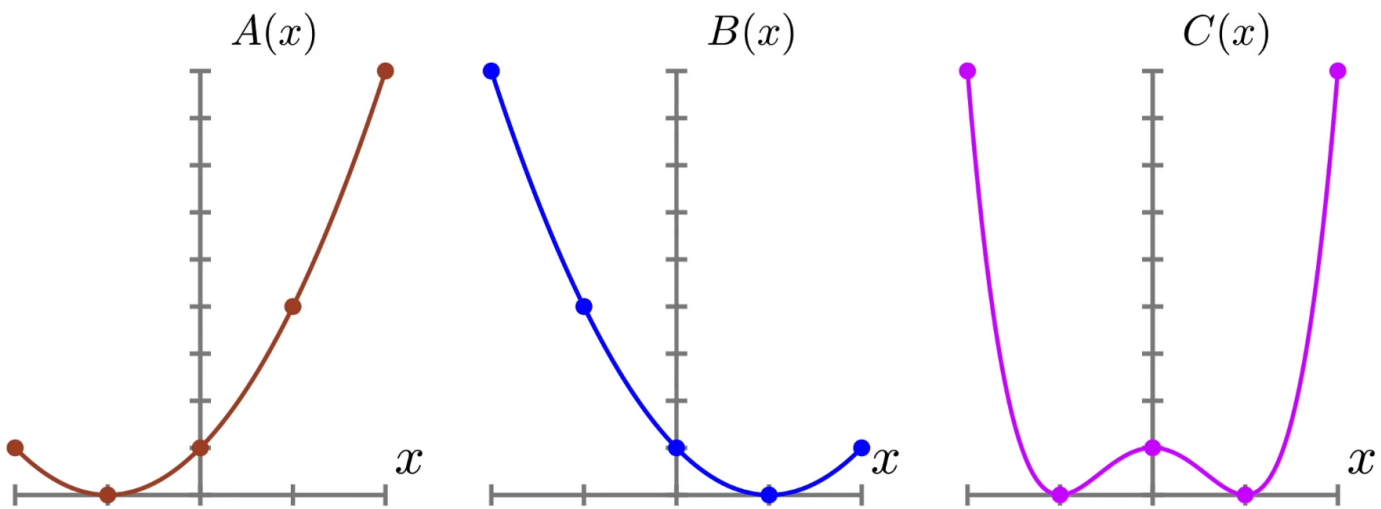
How can this be efficient?!

- Sampling: evaluate the polynomial at $2n+1$ points, $\mathcal{O}(n^2)$
- Pointwise multiplication: $\mathcal{O}(n)$
- Interpolation: up to $\mathcal{O}(n^3)$

$A(x) = x^2 + 2x + 1$ $[(-2,1),(-1,0),(0,1),(1,4),(2,9)]$

$B(x) = x^2 - 2x + 1$ $[(-2,9),(-1,4),(0,1),(1,0),(2,1)]$

$C(x) = x^4 - 2x^2 + 1$ $[(-2,9),(-1,0),(0,1),(1,0),(2,9)]$



The Discrete Fourier Transform (in $\mathbb{C}$)

Sampling

Evaluate the polynomial at the N-th roots of unity $\rightarrow$ take the DFT

$$\mathcal{F}(\omega_k) = \sum_{n=0}^{N} x_n e^{-\frac{2\pi i k}{N} n}$$

Interpolation

Interpolate the points $\rightarrow$ take the inverse DFT

$$\mathcal{F}^{-1}(X_k) = \frac{1}{N} \sum_{n=0}^{N} X_n e^{\frac{2\pi i k}{N} n}$$

Algorithms that calculate the DFT and its inverse are called Fast Fourier Transforms and can achieve $\mathcal{O}(n \log n)$ complexity.

The Discrete Fourier Transform (in $\mathbb{C}$)

Sampling

Evaluate the polynomial at the N-th roots of unity $\rightarrow$ take the DFT

$$\mathcal{F}(\omega_k) = \sum_{n=0}^{N} x_n e^{-\frac{2\pi i k}{N} n}$$

Interpolation

Interpolate the points $\rightarrow$ take the inverse DFT

$$\mathcal{F}^{-1}(X_k) = \frac{1}{N} \sum_{n=0}^{N} X_n e^{\frac{2\pi i k}{N} n}$$

Algorithms that calculate the DFT and its inverse are called Fast Fourier Transforms and can achieve $\mathcal{O}(n \log n)$ complexity.

The Discrete Fourier Transform (in $\mathbb{C}$)

Sampling

Evaluate the polynomial at the N-th roots of unity $\rightarrow$ take the DFT

$$\mathcal{F}(\omega_k) = \sum_{n=0}^{N} x_n e^{-\frac{2\pi i k}{N} n}$$

Interpolation

Interpolate the points $\rightarrow$ take the inverse DFT

$$\mathcal{F}^{-1}(X_k) = \frac{1}{N} \sum_{n=0}^{N} X_n e^{\frac{2\pi i k}{N} n}$$

Algorithms that calculate the DFT and its inverse are called Fast Fourier Transforms and can achieve $\mathcal{O}(n \log n)$ complexity.

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity

Solution: use powers of a primitive element

Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

FFT in finite fields

- $a(x)$ and $b(x)$ are polynomials in $\mathbb{F}_2[x]$
- Each coefficient $a_i, b_i \in \mathbb{F}_2$ is a bit (0, 1)

Problem: Finite fields (of characteristic 2) do not admit roots of unity
Solution: use powers of a primitive element
Caveat(s):

- The degree must be a power of 2: $n = 2^\ell$
- Polynomials require a different representation

Case study: the Lin-Chung-Han FFT [LCH14]

Polynomial evaluation (sampling)

We start from:

- $f(x) = g(X)$, a polynomial of degree 2^m in novelpoly basis
- $\mathbf{W}_m = \mathbb{F}_{2^m}$, the set of points to be evaluated

We want to get:

- $\{f(x)|x \in \mathbf{W}_m\}$, the evaluation of $f(x)$ at the points in $\mathbf{W}_m$

Polynomial evaluation (sampling)

We start from:

- $f(x) = g(X)$, a polynomial of degree 2^m in novelpoly basis
- $\mathbf{W}_m = \mathbb{F}_{2^m}$, the set of points to be evaluated

We want to get:

- $\{f(x)|x \in \mathbf{W}_m\}$, the evaluation of $f(x)$ at the points in $\mathbf{W}_m$

Polynomial evaluation (sampling)

We start from:

- $f(x) = g(X)$, a polynomial of degree 2^m in novelpoly basis
- $\mathbf{W}_m = \mathbb{F}_{2^m}$, the set of points to be evaluated

We want to get:

- $\{f(x)|x \in \mathbf{W}_m\}$, the evaluation of $f(x)$ at the points in $\mathbf{W}_m$

Polynomial evaluation (sampling)

We start from:

- $f(x) = g(X)$, a polynomial of degree 2^m in novelpoly basis
- $\mathbf{W}_m = \mathbb{F}_{2^m}$, the set of points to be evaluated

We want to get:

- $\{f(x)|x \in \mathbf{W}_m\}$, the evaluation of $f(x)$ at the points in $\mathbf{W}_m$

Polynomial evaluation (sampling)

We start from:

- $f(x) = g(X)$, a polynomial of degree 2^m in novelpoly basis
- $\mathbf{W}_m = \mathbb{F}_{2^m}$, the set of points to be evaluated

We want to get:

- $\{f(x) | x \in \mathbf{W}_m\}$, the evaluation of $f(x)$ at the points in $\mathbf{W}_m$

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

The LCH FFT

Divide et impera

Recursively split:

- The polynomial: $f_0(X) + f_1(X)X_{2^k}$
- The set of points: $\mathbf{W}_{k-1}, (\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{w + v_{k-1} | w \in \mathbf{W}_{k-1}\}$

Get 2 for the price of 1

Evaluate f at $\mathbf{W}_{k-1}$; recall that:

- Elements in novelpoly basis are combinations of vanishing polynomials
- Vanishing polynomials are linear: $s_i(x) + s_i(y) = s_i(x + y)$

Thus the evaluations at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ can be obtained starting from the evaluations at $\mathbf{W}_{k-1}$ with a *butterfly* unit

LCH FFT - example

Consider a polynomial f in novelpoly basis of degree greater or equal than $\deg(s_k(x)) = 2^k$ and split it in half:

$$f(x) = p_0(x) + s_k(x)p_1(x)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $\mathbf{W}_{k-1} = \{\alpha \mid \alpha \in \mathbf{W}_{k-1}\}$:

$$p_0(\alpha) + s_k(\alpha)p_1(\alpha)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{\alpha + v_k \mid \alpha \in \mathbf{W}_{k-1}\}$

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k)$$

$p_0(x)$ and $p_1(x)$ are combinations of vanishing polynomials $s_i(x)$ with $i < k$:

$$\sum s_i(\alpha + v_k) = \sum s_i(\alpha) + s_i(v_k) = \sum s_i(\alpha) + 0 = \sum s_i(\alpha)$$

LCH FFT - example

Consider a polynomial f in novelpoly basis of degree greater or equal than $\deg(s_k(x)) = 2^k$ and split it in half:

$$f(x) = p_0(x) + s_k(x)p_1(x)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $\mathbf{W}_{k-1} = \{\alpha \mid \alpha \in \mathbf{W}_{k-1}\}$:

$$p_0(\alpha) + s_k(\alpha)p_1(\alpha)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{\alpha + v_k \mid \alpha \in \mathbf{W}_{k-1}\}$

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k)$$

$p_0(x)$ and $p_1(x)$ are combinations of vanishing polynomials $s_i(x)$ with $i < k$:

$$\sum s_i(\alpha + v_k) = \sum s_i(\alpha) + s_i(v_k) = \sum s_i(\alpha) + 0 = \sum s_i(\alpha)$$

LCH FFT - example

Consider a polynomial f in novelpoly basis of degree greater or equal than $\deg(s_k(x)) = 2^k$ and split it in half:

$$f(x) = p_0(x) + s_k(x)p_1(x)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $\mathbf{W}_{k-1} = \{\alpha \mid \alpha \in \mathbf{W}_{k-1}\}$:

$$p_0(\alpha) + s_k(\alpha)p_1(\alpha)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{\alpha + v_k \mid \alpha \in \mathbf{W}_{k-1}\}$

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k)$$

$p_0(x)$ and $p_1(x)$ are combinations of vanishing polynomials $s_i(x)$ with $i < k$:

$$\sum s_i(\alpha + v_k) = \sum s_i(\alpha) + s_i(v_k) = \sum s_i(\alpha) + 0 = \sum s_i(\alpha)$$

LCH FFT - example

Consider a polynomial f in novelpoly basis of degree greater or equal than $\deg(s_k(x)) = 2^k$ and split it in half:

$$f(x) = p_0(x) + s_k(x)p_1(x)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $\mathbf{W}_{k-1} = \{\alpha \mid \alpha \in \mathbf{W}_{k-1}\}$:

$$p_0(\alpha) + s_k(\alpha)p_1(\alpha)$$

Evaluate $p_0(x)$ and $p_1(x)$ at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1}) = \{\alpha + v_k \mid \alpha \in \mathbf{W}_{k-1}\}$

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k)$$

$p_0(x)$ and $p_1(x)$ are combinations of vanishing polynomials $s_i(x)$ with $i < k$:

$$\sum s_i(\alpha + v_k) = \sum s_i(\alpha) + s_i(v_k) = \sum s_i(\alpha) + 0 = \sum s_i(\alpha)$$

LCH FFT - example

From this consideration, we can observe that:

$$p_0(\alpha + v_k) = p_0(\alpha), \qquad p_1(\alpha + v_k) = p_1(\alpha)$$

Hence we can rewrite the evaluation at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ as:

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k) = p_0(\alpha) + s_k(\alpha)p_1(\alpha) + s_k(v_k)p_1(\alpha)$$

and thus reuse the first evaluation to get the second one.

Figure: Butterfly unit

LCH FFT - example

From this consideration, we can observe that:

$$p_0(\alpha + v_k) = p_0(\alpha), \qquad p_1(\alpha + v_k) = p_1(\alpha)$$

Hence we can rewrite the evaluation at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ as:

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k) = p_0(\alpha) + s_k(\alpha)p_1(\alpha) + s_k(v_k)p_1(\alpha)$$

and thus reuse the first evaluation to get the second one.

Figure: Butterfly unit

LCH FFT - example

From this consideration, we can observe that:

$$p_0(\alpha + v_k) = p_0(\alpha), \qquad p_1(\alpha + v_k) = p_1(\alpha)$$

Hence we can rewrite the evaluation at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ as:

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k) = p_0(\alpha) + s_k(\alpha)p_1(\alpha) + s_k(v_k)p_1(\alpha)$$

and thus reuse the first evaluation to get the second one.

Figure: Butterfly unit

LCH FFT - example

From this consideration, we can observe that:

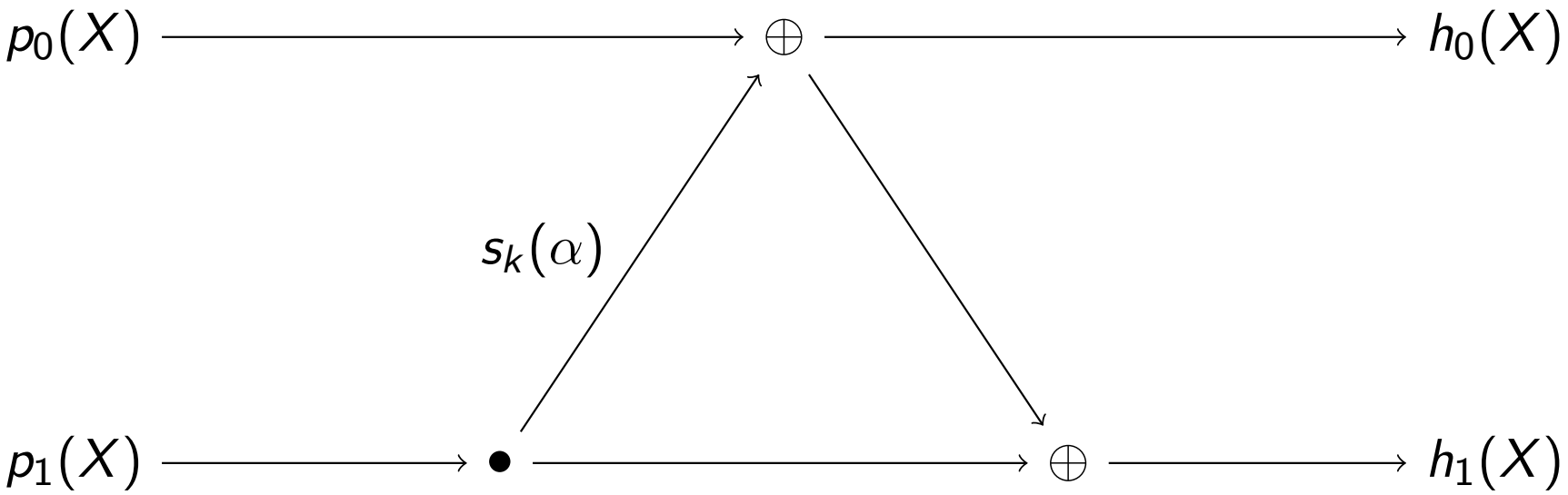
$$p_0(\alpha + v_k) = p_0(\alpha), \qquad p_1(\alpha + v_k) = p_1(\alpha)$$

Hence we can rewrite the evaluation at $(\mathbf{W}_k \setminus \mathbf{W}_{k-1})$ as:

$$p_0(\alpha + v_k) + s_k(\alpha + v_k)p_1(\alpha + v_k) = p_0(\alpha) + s_k(\alpha)p_1(\alpha) + s_k(v_k)p_1(\alpha)$$

and thus reuse the first evaluation to get the second one.

Figure: Butterfly unit



Frobenius map

We can use the Frobenius map

$$\phi : \mathbb{F}_2 \to \mathbb{F}_2 \qquad x \mapsto x^2$$

to derive more values of a polynomial f of degree n from less evaluations.

Frobenius partition

We can build a set of points $\mathbf{S} \subset \mathbb{F}_{2^m}$ as:

$$\mathbf{S} := v_{l+m/2} + \mathbf{W}_l = \{v_{l+m/2} + x \mid x \in \mathbf{W}_l\}$$

which has size $n' = |\mathbf{S}| = |\mathbf{W}_l| = n/m$. By repeatedly applying ϕ to $\mathbf{S}$ we get

$$\mathbf{S} \cup \phi(\mathbf{S}) \cup \cdots \cup \phi^{m-1}(\mathbf{S}) = \mathbf{O}$$

with $|\mathbf{O}| = n$.

Frobenius map

We can use the Frobenius map

$$\phi : \mathbb{F}_2 \to \mathbb{F}_2 \qquad x \mapsto x^2$$

to derive more values of a polynomial f of degree n from less evaluations.

Frobenius partition

We can build a set of points $\mathbf{S} \subset \mathbb{F}_{2^m}$ as:

$$\mathbf{S} := v_{l+m/2} + \mathbf{W}_l = \{v_{l+m/2} + x \mid x \in \mathbf{W}_l\}$$

which has size $n' = |\mathbf{S}| = |\mathbf{W}_l| = n/m$. By repeatedly applying ϕ to $\mathbf{S}$ we get

$$\mathbf{S} \cup \phi(\mathbf{S}) \cup \cdots \cup \phi^{m-1}(\mathbf{S}) = \mathbf{O}$$

with $|\mathbf{O}| = n$.

Frobenius map

We can use the Frobenius map

$$\phi : \mathbb{F}_2 \to \mathbb{F}_2 \qquad x \mapsto x^2$$

to derive more values of a polynomial f of degree n from less evaluations.

Frobenius partition

We can build a set of points $\mathbf{S} \subset \mathbb{F}_{2^m}$ as:

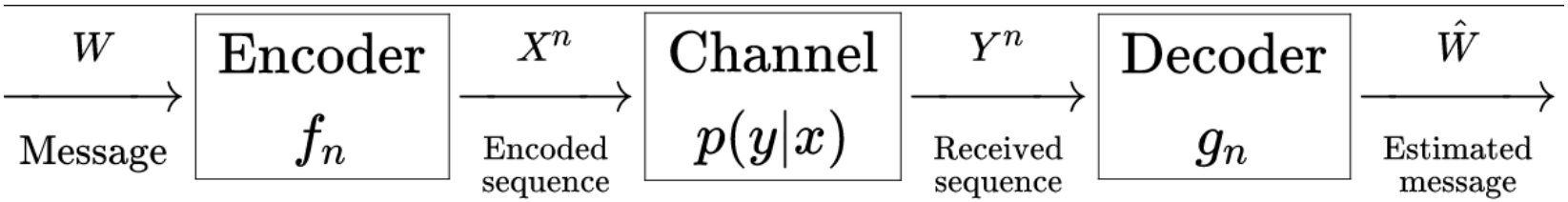
$$\mathbf{S} := v_{l+m/2} + \mathbf{W}_l = \{v_{l+m/2} + x \mid x \in \mathbf{W}_l\}$$

which has size $n' = |\mathbf{S}| = |\mathbf{W}_l| = n/m$. By repeatedly applying ϕ to $\mathbf{S}$ we get

$$\mathbf{S} \cup \phi(\mathbf{S}) \cup \cdots \cup \phi^{m-1}(\mathbf{S}) = \mathbf{O}$$

with $|\mathbf{O}| = n$.

Error Correction Codes

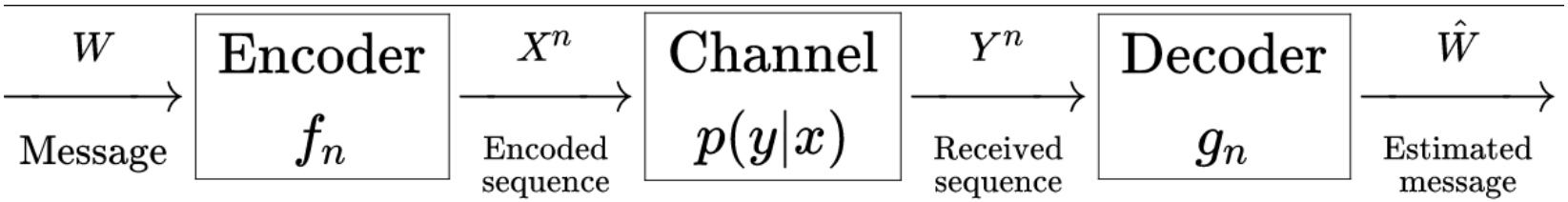


Problem: we want to send a message m over a noisy channel.

- The channel is noisy: it introduces errors in the message
- The receiver wants to recover the original message

Solution: send, along with the message, information that allows the receiver to correct the errors.

Error Correction Codes

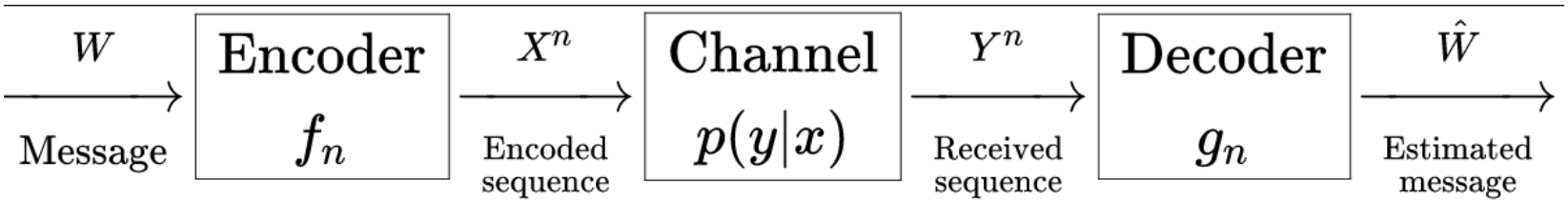


Problem: we want to send a message m over a noisy channel.

- The channel is noisy: it introduces errors in the message
- The receiver wants to recover the original message

Solution: send, along with the message, information that allows the receiver to correct the errors.

Error Correction Codes

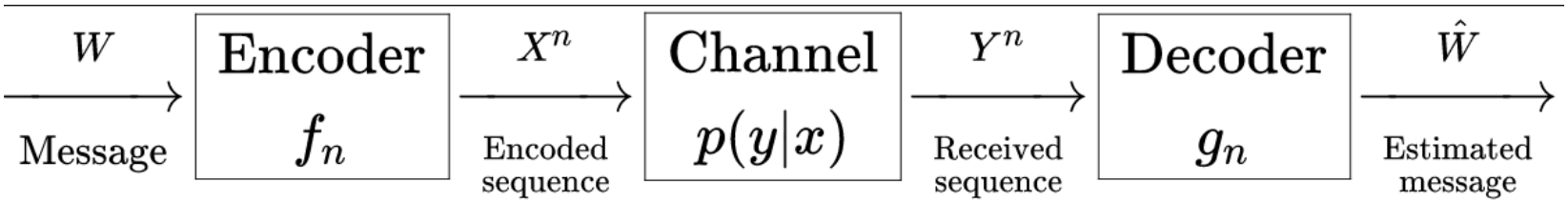


Problem: we want to send a message m over a noisy channel.

- The channel is noisy: it introduces errors in the message
- The receiver wants to recover the original message

Solution: send, along with the message, information that allows the receiver to correct the errors.

Error Correction Codes



Problem: we want to send a message m over a noisy channel.

- The channel is noisy: it introduces errors in the message
- The receiver wants to recover the original message

Solution: send, along with the message, information that allows the receiver to correct the errors.

Linear codes

Let $\mathcal{V}$ be a vector space of dimension n over $\mathbb{F}_2$. A linear code $\mathcal{C} = [n, k]$ is a subspace of $\mathcal{V}$ of dimension k.

- Messages are k bits long
- We add $n - k$ bits to detect and correct errors (up to δ)

Generator Matrix

The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG} \mid \mathbf{m} \in \mathbb{F}_2^k\}$$

Linear codes

Let $\mathcal{V}$ be a vector space of dimension n over $\mathbb{F}_2$. A linear code $\mathcal{C} = [n, k]$ is a subspace of $\mathcal{V}$ of dimension k.

- Messages are k bits long
- We add $n - k$ bits to detect and correct errors (up to δ)

Generator Matrix

The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG} \mid \mathbf{m} \in \mathbb{F}_2^k\}$$

Linear codes

Let $\mathcal{V}$ be a vector space of dimension n over $\mathbb{F}_2$. A linear code $\mathcal{C} = [n, k]$ is a subspace of $\mathcal{V}$ of dimension k.

- Messages are k bits long
- We add $n - k$ bits to detect and correct errors (up to δ)

Generator Matrix

The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG} \mid \mathbf{m} \in \mathbb{F}_2^k\}$$

Syndrome

Parity Check Matrix

The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ is the parity check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{v}^\top = 0\}$$

The syndrome of a word $\mathbf{v} \in \mathcal{C}$ is defined as:

$$\mathbf{S}(\mathbf{v}) = \mathbf{H}\mathbf{v}^\top$$

This definition is equivalent to the one based on the generator matrix.

Syndrome

Parity Check Matrix

The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ is the parity check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{v}^\top = 0\}$$

The syndrome of a word $\mathbf{v} \in \mathcal{C}$ is defined as:

$$\mathbf{S}(\mathbf{v}) = \mathbf{H}\mathbf{v}^\top$$

This definition is equivalent to the one based on the generator matrix.

Syndrome

Parity Check Matrix

The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ is the parity check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{v}^\top = 0\}$$

The syndrome of a word $\mathbf{v} \in \mathcal{C}$ is defined as:

$$\mathbf{S}(\mathbf{v}) = \mathbf{H}\mathbf{v}^\top$$

This definition is equivalent to the one based on the generator matrix.

Syndrome Decoding

Given an encoded message, find the closest message w.r.t. a certain metric (*i.e.* Hamming weight, $\mathtt{wt}()$).

How hard can this be?

- Take a word $\mathbf{x}$ with $\mathtt{wt}(w)$
- Calculate the syndrome $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$
- Given $\mathbf{H}$ and $\mathbf{y}^\top$, retrieve $\mathbf{x}$

This problem is **NP-complete** [BMvT78].

Syndrome Decoding

Given an encoded message, find the closest message w.r.t. a certain metric (*i.e.* Hamming weight, $\mathtt{wt}()$).

How hard can this be?

- Take a word $\mathbf{x}$ with $\mathtt{wt}(w)$
- Calculate the syndrome $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$
- Given $\mathbf{H}$ and $\mathbf{y}^\top$, retrieve $\mathbf{x}$

This problem is **NP-complete** [BMvT78].

Syndrome Decoding

Given an encoded message, find the closest message w.r.t. a certain metric (*i.e.* Hamming weight, $\mathtt{wt}()$).

How hard can this be?

- Take a word $\mathbf{x}$ with $\mathtt{wt}(w)$
- Calculate the syndrome $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$
- Given $\mathbf{H}$ and $\mathbf{y}^\top$, retrieve $\mathbf{x}$

This problem is **NP-complete** [BMvT78].

Syndrome Decoding

Given an encoded message, find the closest message w.r.t. a certain metric (*i.e.* Hamming weight, $\mathtt{wt}()$).

How hard can this be?

- Take a word $\mathbf{x}$ with $\mathtt{wt}(w)$
- Calculate the syndrome $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$
- Given $\mathbf{H}$ and $\mathbf{y}^\top$, retrieve $\mathbf{x}$

This problem is **NP-complete** [BMvT78].

Syndrome Decoding

Given an encoded message, find the closest message w.r.t. a certain metric (*i.e.* Hamming weight, $\mathtt{wt}()$).

How hard can this be?

- Take a word $\mathbf{x}$ with $\mathtt{wt}(w)$
- Calculate the syndrome $\mathbf{y}^\top = \mathbf{H}\mathbf{x}^\top$
- Given $\mathbf{H}$ and $\mathbf{y}^\top$, retrieve $\mathbf{x}$

This problem is **NP-complete** [BMvT78].

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Hamming Quasi-Cyclic

- IND-CPA Public Key Encryption
- IND-CCA2 Key Encapsulation Mechanism
- Based on a variant of the syndrome decoding problem
- Employs quasi-cyclic codes to shorten key sizes

Codes

HQC employs two codes:

- a decodable $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ (public)
- a random $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I}_n | \mathbf{circ}(\mathbf{h}))$

Public Key Encryption

Setup

Generate the parameters $(n, k, \delta, w, w_r, w_e)$ for the corresponding security level λ

Key Generation

- Generate $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2^n$
- Generate $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = w$
- Calculate $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$

$$\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \qquad \mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$$

Public Key Encryption

Setup

Generate the parameters $(n, k, \delta, w, w_r, w_e)$ for the corresponding security level λ

Key Generation

- Generate $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2^n$
- Generate $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = w$
- Calculate $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$

$$\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \qquad \mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$$

Public Key Encryption

Setup

Generate the parameters $(n, k, \delta, w, w_r, w_e)$ for the corresponding security level λ

Key Generation

- Generate $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2^n$
- Generate $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = w$
- Calculate $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$

$$\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \qquad \mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$$

Public Key Encryption

Setup

Generate the parameters $(n, k, \delta, w, w_r, w_e)$ for the corresponding security level λ

Key Generation

- Generate $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2^n$
- Generate $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = w$
- Calculate $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$

$$\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \qquad \mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$$

Public Key Encryption

Setup

Generate the parameters $(n, k, \delta, w, w_r, w_e)$ for the corresponding security level λ

Key Generation

- Generate $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2^n$
- Generate $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = w$
- Calculate $\mathbf{s} = \mathbf{x} + \mathbf{hy}$

$$\mathcal{K}_{pub} = (\mathbf{h}, \mathbf{s}) \qquad \mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$$

Public Key Encryption

To encrypt a message $\mathbf{m} \in \mathbb{F}_2^k$:

Encryption

- Generate $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{r_1}) = \mathtt{wt}(\mathbf{r_2}) = w_r$
- Generate $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{e}) = w_e$
- Calculate $\mathbf{u} = \mathbf{r_1} + \mathbf{h}\mathbf{r_2}$
- Calculate $\mathbf{v} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r_2} + \mathbf{e}$

$$\mathrm{ctx} = (\mathbf{u}, \mathbf{v})$$

Public Key Encryption

To encrypt a message $\mathbf{m} \in \mathbb{F}_2^k$:

Encryption

- Generate $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{r_1}) = \mathtt{wt}(\mathbf{r_2}) = w_r$
- Generate $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{e}) = w_e$
- Calculate $\mathbf{u} = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$
- Calculate $\mathbf{v} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e}$

$$\mathrm{ctx} = (\mathbf{u}, \mathbf{v})$$

Public Key Encryption

To encrypt a message $\mathbf{m} \in \mathbb{F}_2^k$:

Encryption

- Generate $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{r_1}) = \mathtt{wt}(\mathbf{r_2}) = w_r$
- Generate $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{e}) = w_e$
- Calculate $\mathbf{u} = \mathbf{r_1} + \mathbf{h}\mathbf{r_2}$
- Calculate $\mathbf{v} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r_2} + \mathbf{e}$

$$\mathrm{ctx} = (\mathbf{u}, \mathbf{v})$$

Public Key Encryption

To encrypt a message $\mathbf{m} \in \mathbb{F}_2^k$:

Encryption

- Generate $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{r_1}) = \mathtt{wt}(\mathbf{r_2}) = w_r$
- Generate $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n$, with $\mathtt{wt}(\mathbf{e}) = w_e$
- Calculate $\mathbf{u} = \mathbf{r_1} + \mathbf{h}\mathbf{r_2}$
- Calculate $\mathbf{v} = \mathbf{m}\mathbf{G} + \mathbf{s}\mathbf{r_2} + \mathbf{e}$

$$\mathrm{ctx} = (\mathbf{u}, \mathbf{v})$$

Public Key Encryption

Decryption

From the ctx $= (\mathbf{u}, \mathbf{v})$ and the private key $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$:

$$\begin{aligned}\mathbf{v} - \mathbf{u} \cdot \mathbf{y} &= (\mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}) - (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2)\mathbf{y} = \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} \\ &= \mathbf{mG} + (\mathbf{x} + \mathbf{h} \cdot \mathbf{y})\mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 \cdot \mathbf{y} + \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y}) \\ &= \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y} - \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y} \\ &= \mathbf{mG} + \mathbf{x}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}\end{aligned}$$

So we have $\mathbf{m} = \mathcal{C}.\text{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$ whenever

$$\mathtt{wt}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}) \leq \delta$$

Public Key Encryption

Decryption

From the ctx $= (\mathbf{u}, \mathbf{v})$ and the private key $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$:

$$\begin{aligned}\mathbf{v} - \mathbf{u} \cdot \mathbf{y} &= (\mathbf{mG} + \mathbf{s} \cdot \mathbf{r_2} + \mathbf{e}) - (\mathbf{r_1} + \mathbf{h} \cdot \mathbf{r_2})\mathbf{y} = \mathbf{mG} + \mathbf{x} \cdot \mathbf{r_2} - \mathbf{r_1} \cdot \mathbf{y} + \mathbf{e} \\ &= \mathbf{mG} + (\mathbf{x} + \mathbf{h} \cdot \mathbf{y})\mathbf{r_2} + \mathbf{e} - (\mathbf{r_1} \cdot \mathbf{y} + \mathbf{h} \cdot \mathbf{r_2} \cdot \mathbf{y}) \\ &= \mathbf{mG} + \mathbf{x} \cdot \mathbf{r_2} + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r_2} + \mathbf{e} - \mathbf{r_1} \cdot \mathbf{y} - \mathbf{h} \cdot \mathbf{r_2} \cdot \mathbf{y} \\ &= \mathbf{mG} + \mathbf{x_2} - \mathbf{r_1} \cdot \mathbf{y} + \mathbf{e}\end{aligned}$$

So we have $\mathbf{m} = \mathcal{C}.\text{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$ whenever

$$\mathtt{wt}(\mathbf{x} \cdot \mathbf{r_2} - \mathbf{r_1} \cdot \mathbf{y} + \mathbf{e}) \leq \delta$$

Public Key Encryption

Decryption

From the ctx $= (\mathbf{u}, \mathbf{v})$ and the private key $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$:

$$\begin{aligned}\mathbf{v} - \mathbf{u} \cdot \mathbf{y} &= (\mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}) - (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2)\mathbf{y} = \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} \\ &= \mathbf{mG} + (\mathbf{x} + \mathbf{h} \cdot \mathbf{y})\mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 \cdot \mathbf{y} + \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y}) \\ &= \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y} - \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y} \\ &= \mathbf{mG} + \mathbf{x}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}\end{aligned}$$

So we have $\mathbf{m} = \mathcal{C}.\text{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$ whenever

$$\mathtt{wt}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}) \leq \delta$$

Public Key Encryption

Decryption

From the ctx $= (\mathbf{u}, \mathbf{v})$ and the private key $\mathcal{K}_{pri} = (\mathbf{x}, \mathbf{y})$:

$$\begin{aligned}\mathbf{v} - \mathbf{u} \cdot \mathbf{y} &= (\mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}) - (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2)\mathbf{y} = \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e} \\ &= \mathbf{mG} + (\mathbf{x} + \mathbf{h} \cdot \mathbf{y})\mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 \cdot \mathbf{y} + \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y}) \\ &= \mathbf{mG} + \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{h} \cdot \mathbf{y} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y} - \mathbf{h} \cdot \mathbf{r}_2 \cdot \mathbf{y} \\ &= \mathbf{mG} + \mathbf{x}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}\end{aligned}$$

So we have $\mathbf{m} = \mathcal{C}.\text{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$ whenever

$$\mathtt{wt}(\mathbf{x} \cdot \mathbf{r}_2 - \mathbf{r}_1 \cdot \mathbf{y} + \mathbf{e}) \leq \delta$$

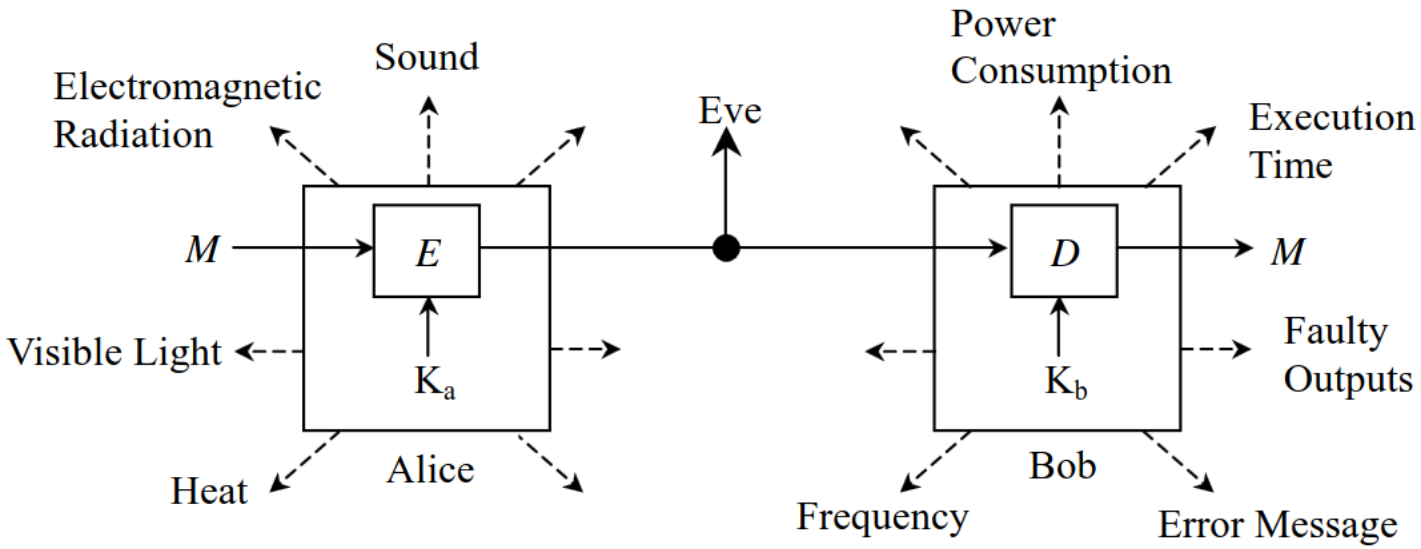
Key Encapsulation Mechanism

By applying the Fujisaki-Okamoto transformation it is possible to build a IND-CCA2 KEM. Let $\mathcal{G}, \mathcal{H}, \mathcal{K}$ be hash functions and $\mathcal{E}$ an instance of HQC.PKE

- **Setup**: generate the parameters λ as in HQC.PKE, except that k will be the length of the key to be exchanged
- **KeyGen**: as in HQC.PKE

Side channel attacks

Devices running cryptosystems may expose secrets through *side channels*
Observable aspects of the system from which information can be extracted



Timing attacks

Execution time may vary based on the inputs

- Loops iterating over the input
- Branches based on the input

Countermeasures

Don't use sensitive inputs in branching instructions

- Use constant time algorithms

Timing attacks

Execution time may vary based on the inputs

- Loops iterating over the input
- Branches based on the input

Countermeasures

Don't use sensitive inputs in branching instructions

- Use constant time algorithms

Timing attacks

Execution time may vary based on the inputs

- Loops iterating over the input
- Branches based on the input

Countermeasures

Don't use sensitive inputs in branching instructions

- Use constant time algorithms

Timing attacks

Execution time may vary based on the inputs

- Loops iterating over the input
- Branches based on the input

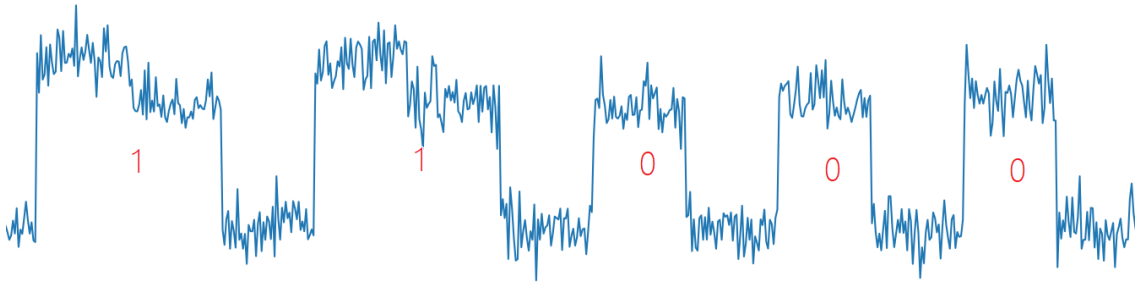
Countermeasures

Don't use sensitive inputs in branching instructions

- Use constant time algorithms

Power attacks

Different operations require different amounts of power
Applies to both power and EM attacks



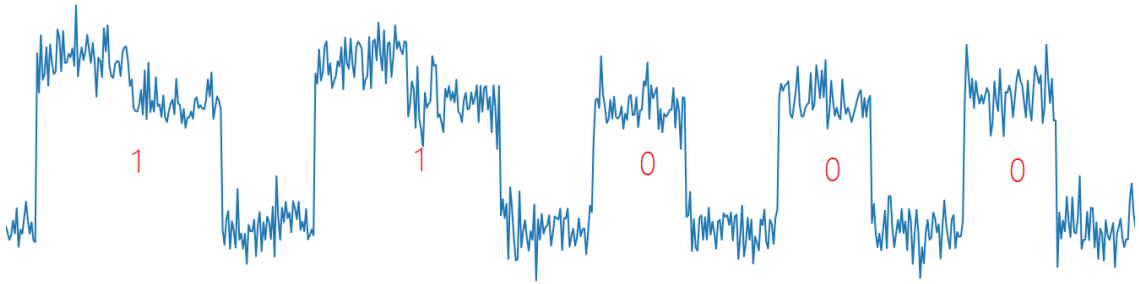
Countermeasures

Mask sensitive data with random values

- Makes the power consumption fluctuate randomly

Power attacks

Different operations require different amounts of power
Applies to both power and EM attacks



Countermeasures

Mask sensitive data with random values

- Makes the power consumption fluctuate randomly

Implementation

Parameters

Our target: `hqc-128` on ARM Cortex-M4

Multiplication parameters

- Maximum polynomial degree: $n = 17669$
- Hamming weight of sparse polynomials: $\omega = 75$

Target board

STM32L476RG

- 32-bit CPU
- 80 MHz
- 128 KB RAM
- 1 MB Flash

Parameters

Our target: `hqc-128` on ARM Cortex-M4

Multiplication parameters

- Maximum polynomial degree: $n = 17669$
- Hamming weight of sparse polynomials: $\omega = 75$

Target board

STM32L476RG

- 32-bit CPU
- 80 MHz
- 128 KB RAM
- 1 MB Flash

Parameters

Our target: `hqc-128` on ARM Cortex-M4

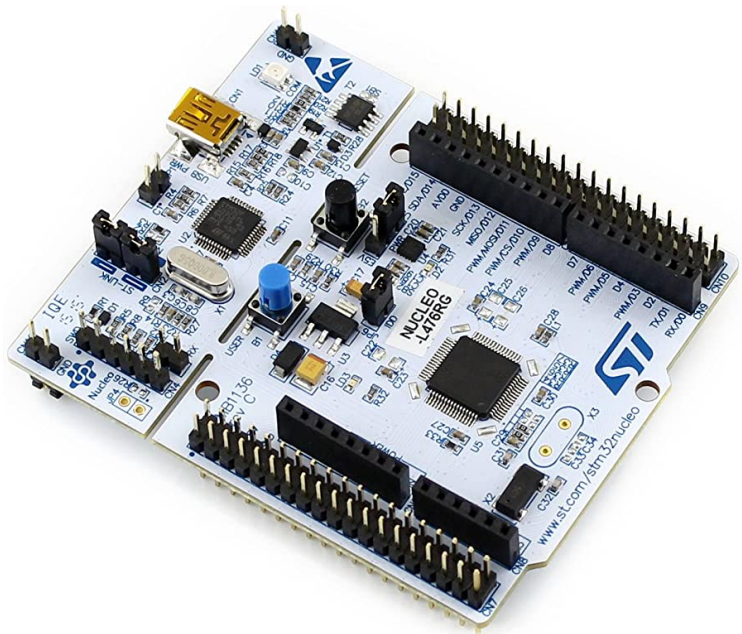
Multiplication parameters

- Maximum polynomial degree: $n = 17669$
- Hamming weight of sparse polynomials: $\omega = 75$

Target board

STM32L476RG

- 32-bit CPU
- 80 MHz
- 128 KB RAM
- 1 MB Flash



Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Polynomial representation

We consider the ring of binary polynomials $\mathcal{R} = \mathbb{F}_2[x]/\langle x^n + 1 \rangle$

We have two ways of representing binary polynomials:

- sparse: a vector of indexes, every index is a non-null coefficient
- dense: a vector of coefficients, every entry is a coefficient

Example

The polynomial $f(x) = x^{15} + x^{13} + x^8 + x^6 + x + 1$ can be represented as:

- sparse: $[15, 13, 8, 6, 1, 0]$
- dense: $[1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1]$

Memory usage - 32 bit architectures

- sparse: one integer per coefficient (*i.e.* 4 bytes per coefficient)
- dense: one bit per coefficient (*i.e.* 0.25 bytes per coefficient)

Shift&Add

Multiplying a dense polynomial by x^i is equivalent to a circular shift ($\lll_n$) of i positions to the left:

- $a(x)$ in dense representation
- $b(x) = [b_0, \cdots, b_{\omega-1}]$ in sparse representation
- Calculate $c_i(x) \leftarrow a(x) \lll_n b_i$
- Sum the partial results $c(x) = \sum_{i=0}^{\omega-1} c_i(x)$

Pros and cons

- Extremely fast for $\omega \ll n$
- Non constant-time execution

Shift&Add

Multiplying a dense polynomial by x^i is equivalent to a circular shift ($\lll_n$) of i positions to the left:

- $a(x)$ in dense representation
- $b(x) = [b_0, \cdots, b_{\omega-1}]$ in sparse representation
- Calculate $c_i(x) \leftarrow a(x) \lll_n b_i$
- Sum the partial results $c(x) = \sum_{i=0}^{\omega-1} c_i(x)$

Pros and cons

- Extremely fast for $\omega \ll n$
- Non constant-time execution

Shift&Add

Multiplying a dense polynomial by x^i is equivalent to a circular shift ($\lll_n$) of i positions to the left:

- $a(x)$ in dense representation
- $b(x) = [b_0, \cdots, b_{\omega-1}]$ in sparse representation
- Calculate $c_i(x) \leftarrow a(x) \lll_n b_i$
- Sum the partial results $c(x) = \sum_{i=0}^{\omega-1} c_i(x)$

Pros and cons

- Extremely fast for $\omega \ll n$
- Non constant-time execution

Shift&Add

Multiplying a dense polynomial by x^i is equivalent to a circular shift ($\lll_n$) of i positions to the left:

- $a(x)$ in dense representation
- $b(x) = [b_0, \cdots, b_{\omega-1}]$ in sparse representation
- Calculate $c_i(x) \leftarrow a(x) \lll_n b_i$
- Sum the partial results $c(x) = \sum_{i=0}^{\omega-1} c_i(x)$

Pros and cons

- Extremely fast for $\omega \ll n$
- Non constant-time execution

Shift&Add

Multiplying a dense polynomial by x^i is equivalent to a circular shift ($\lll_n$) of i positions to the left:

- $a(x)$ in dense representation
- $b(x) = [b_0, \cdots, b_{\omega-1}]$ in sparse representation
- Calculate $c_i(x) \leftarrow a(x) \lll_n b_i$
- Sum the partial results $c(x) = \sum_{i=0}^{\omega-1} c_i(x)$

Pros and cons

- Extremely fast for $\omega \ll n$
- Non constant-time execution

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \ll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Constant time S&A

Calculate all the shifts, but only sum the ones that are needed:

Monomial-Polynomial multiplication

Let $a(x)$ be a dense polynomial and x^b a monomial, with $b = (\beta_{m-1}\beta_{m-2}\cdots\beta_0)_{\texttt{bin}}$. For each bit $i \in \{0, \cdots, m-1\}$, we have:

- $tmp(x) \leftarrow a(x) \lll_n i$
- if $\beta_i = 1$ then $\texttt{mask} = [111\cdots11]_n$, otherwise $\texttt{mask} = [000\cdots00]_n$
- $c(x) \leftarrow c(x) + ((tmp(x) \wedge \texttt{mask}))$
  - $\beta_i = 1$: $c(x) \leftarrow c(x) + tmp(x)$
  - $\beta_i = 0$: $c(x) \leftarrow c(x) + [000\cdots00]_n$

Pros and cons

- Constant-time execution
- Slower than plain S&A

Frobenius Additive FFT

Based on the BIKE optimized implementation on ARM Cortex-M4 [CCK21]

Let $a(x)$ and $b(x)$ be two dense polynomials of degree n a power of 2

- Basis conversion (from monomial to novelpoly)
- Evaluation with the FFT
  - Encode (truncated FFT)
  - Butterflies
- Pointwise multiplication
- Inverse FFT
- Basis conversion (from novelpoly to monomial)

Inverse operations follow the same principles of the forward ones.

Operands have fixed sizes $\rightarrow$ unroll and optimize recursive calls

Basis conversion

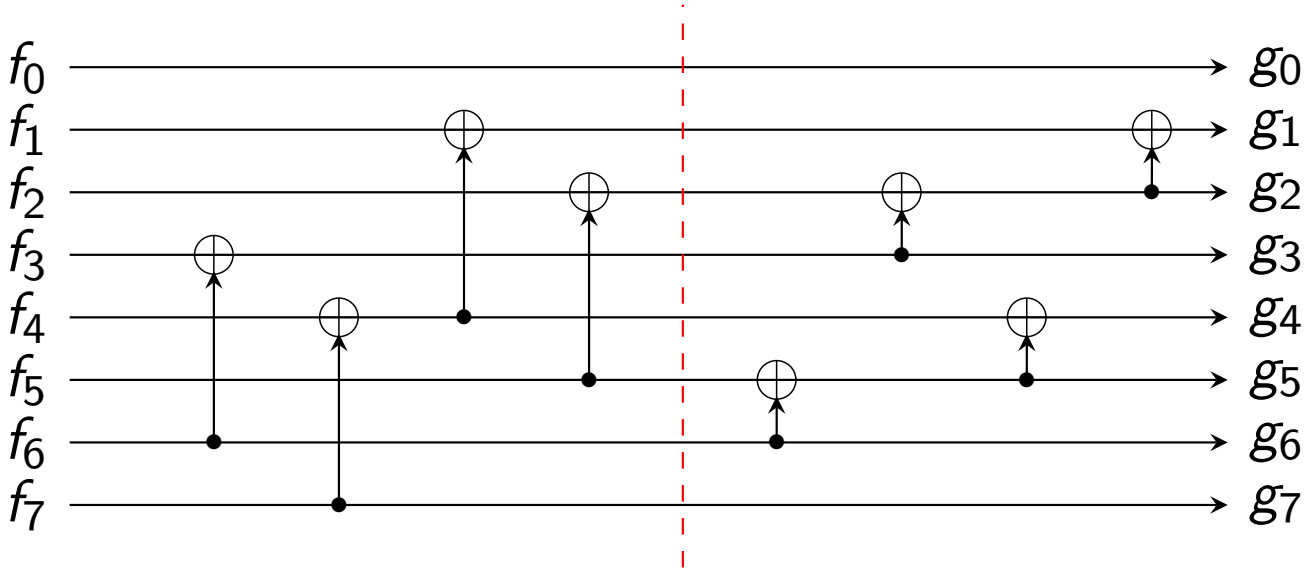
Recursively split the polynomial into halves by dividing by $s_i(x)$

No actual division required, just XORs:

Also, skip the (padded) null coefficients

Basis conversion

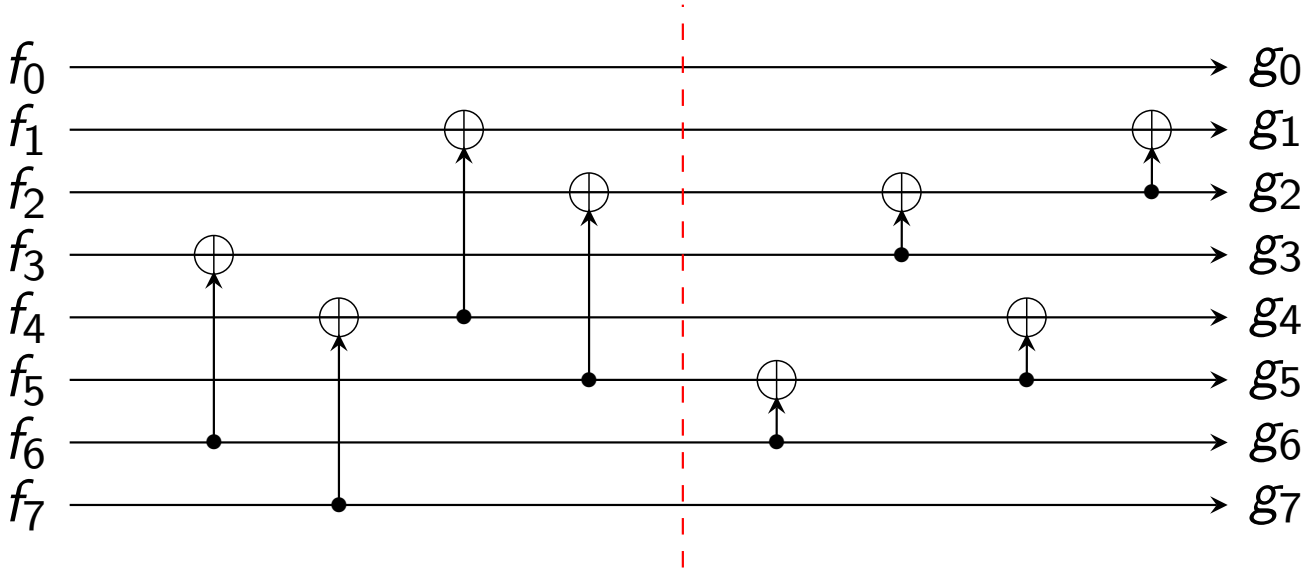
Recursively split the polynomial into halves by dividing by $s_i(x)$
No actual division required, just XORs:



Also, skip the (padded) null coefficients

Basis conversion

Recursively split the polynomial into halves by dividing by $s_i(x)$
No actual division required, just XORs:



Also, skip the (padded) null coefficients

Evaluation with the FFT

Frobenius partition

Polynomial degree $2 \cdot 17669 \rightarrow n = 65536$

32-bit words $\rightarrow m = 32$

Optimal Frobenius partition:

$$\mathbf{S} = v_{l+m/2} + \mathbf{W}_l = v_{27} + \mathbf{W}_{11}$$

with $l = \log_2(n/m)$

Encoding

The first $\log_2(m)$ layers of the FFT use multipliers depending only on m

- Encode (truncated FFT) the first $\log_2(m)$ recursion layers
- *Classic* butterflies for the remaining layers

Evaluation with the FFT

Frobenius partition

Polynomial degree $2 \cdot 17669 \rightarrow n = 65536$

32-bit words $\rightarrow m = 32$

Optimal Frobenius partition:

$$\mathbf{S} = v_{l+m/2} + \mathbf{W}_l = v_{27} + \mathbf{W}_{11}$$

with $l = \log_2(n/m)$

Encoding

The first $\log_2(m)$ layers of the FFT use multipliers depending only on m

- Encode (truncated FFT) the first $\log_2(m)$ recursion layers
- *Classic* butterflies for the remaining layers

Evaluation with the FFT

Frobenius partition

Polynomial degree $2 \cdot 17669 \rightarrow n = 65536$

32-bit words $\rightarrow m = 32$

Optimal Frobenius partition:

$$\mathbf{S} = v_{l+m/2} + \mathbf{W}_l = v_{27} + \mathbf{W}_{11}$$

with $l = \log_2(n/m)$

Encoding

The first $\log_2(m)$ layers of the FFT use multipliers depending only on m

- Encode (truncated FFT) the first $\log_2(m)$ recursion layers
- *Classic* butterflies for the remaining layers

Evaluation with the FFT

Frobenius partition

Polynomial degree $2 \cdot 17669 \rightarrow n = 65536$

32-bit words $\rightarrow m = 32$

Optimal Frobenius partition:

$$\mathbf{S} = v_{l+m/2} + \mathbf{W}_l = v_{27} + \mathbf{W}_{11}$$

with $l = \log_2(n/m)$

Encoding

The first $\log_2(m)$ layers of the FFT use multipliers depending only on m

- Encode (truncated FFT) the first $\log_2(m)$ recursion layers
- *Classic* butterflies for the remaining layers

Evaluation with the FFT

Frobenius partition

Polynomial degree $2 \cdot 17669 \rightarrow n = 65536$

32-bit words $\rightarrow m = 32$

Optimal Frobenius partition:

$$\mathbf{S} = v_{l+m/2} + \mathbf{W}_l = v_{27} + \mathbf{W}_{11}$$

with $l = \log_2(n/m)$

Encoding

The first $\log_2(m)$ layers of the FFT use multipliers depending only on m

- Encode (truncated FFT) the first $\log_2(m)$ recursion layers
- *Classic* butterflies for the remaining layers

Encoding

The first $l_m = \log_2(m)$ layers of the FFT convert elements from $\mathbb{F}_2^n$ to $\mathbb{F}_{2^m}^{n'}$.

The twiggle factors can be rewritten as

$$s_{l+1}(v_{l+m/2}), \cdots, s_{l+l_m}(v_{l+m/2})$$

$$v_{m/2-1}, \cdots, v_{m/2-l_m}$$

so the encoding can be implemented as a linear map:

$$g_i = \sum_{j=0}^{m-1} f_{j \cdot n' + i} \cdot r_j = \begin{bmatrix} f_{0 \cdot n'} & f_{1 \cdot n'} & \cdots & f_{(m-1) \cdot n'} \end{bmatrix} \cdot \begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_{m-1} \end{bmatrix}$$

Encoding

The first $l_m = \log_2(m)$ layers of the FFT convert elements from $\mathbb{F}_2^n$ to $\mathbb{F}_{2^m}^{n'}$. The twiggle factors can be rewritten as

$$s_{l+1}(v_{l+m/2}), \cdots, s_{l+l_m}(v_{l+m/2})$$

$$v_{m/2-1}, \cdots, v_{m/2-l_m}$$

so the encoding can be implemented as a linear map:

$$g_i = \sum_{j=0}^{m-1} f_{j \cdot n' + i} \cdot r_j = \begin{bmatrix} f_{0 \cdot n'} & f_{1 \cdot n'} & \cdots & f_{(m-1) \cdot n'} \end{bmatrix} \cdot \begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_{m-1} \end{bmatrix}$$

Encoding

The first $l_m = \log_2(m)$ layers of the FFT convert elements from $\mathbb{F}_2^n$ to $\mathbb{F}_{2^m}^{n'}$. The twiggle factors can be rewritten as

$$s_{l+1}(v_{l+m/2}), \cdots, s_{l+l_m}(v_{l+m/2})$$

$$v_{m/2-1}, \cdots, v_{m/2-l_m}$$

so the encoding can be implemented as a linear map:

$$g_i = \sum_{j=0}^{m-1} f_{j \cdot n' + i} \cdot r_j = \begin{bmatrix} f_{0 \cdot n'} & f_{1 \cdot n'} & \cdots & f_{(m-1) \cdot n'} \end{bmatrix} \cdot \begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_{m-1} \end{bmatrix}$$

Multiplications

Butterfly twiggle factors

Multiplication step: $\beta \cdot p_1(s_1(\beta))$, $\beta = (v + w) \in v_{l+m/2-i} + \mathbf{W}_{l-i}$

- Precompute and store all the β values
- Notice that $\gamma \cdot \beta = \gamma \cdot (v + w) = \gamma \cdot v + \gamma \cdot w$
  - $\gamma \cdot v$ can be computed with a binary circuit (v are fixed)
  - $\gamma \cdot w$ can be computed in a subfield to reduce the number of bits

Pointwise multiplications

The resulting evaluation points are, by construction, in $\mathbb{F}_{2^{32}}$.
Apply Karatsuba (only once) to operate in $\mathbb{F}_{2^{16}}$ and have the results fit in the 32-bit CPU registers.

Multiplications

Butterfly twiggle factors

Multiplication step: $\beta \cdot p_1(s_1(\beta))$, $\beta = (v + w) \in v_{l+m/2-i} + \mathbf{W}_{l-i}$

- Precompute and store all the β values
- Notice that $\gamma \cdot \beta = \gamma \cdot (v + w) = \gamma \cdot v + \gamma \cdot w$
  - $\gamma \cdot v$ can be computed with a binary circuit (v are fixed)
  - $\gamma \cdot w$ can be computed in a subfield to reduce the number of bits

Pointwise multiplications

The resulting evaluation points are, by construction, in $\mathbb{F}_{2^{32}}$.
Apply Karatsuba (only once) to operate in $\mathbb{F}_{2^{16}}$ and have the results fit in the 32-bit CPU registers.

Multiplications

Butterfly twiggle factors

Multiplication step: $\beta \cdot p_1(s_1(\beta))$, $\beta = (v + w) \in v_{l+m/2-i} + \mathbf{W}_{l-i}$

- Precompute and store all the β values
- Notice that $\gamma \cdot \beta = \gamma \cdot (v + w) = \gamma \cdot v + \gamma \cdot w$
  - $\gamma \cdot v$ can be computed with a binary circuit (v are fixed)
  - $\gamma \cdot w$ can be computed in a subfield to reduce the number of bits

Pointwise multiplications

The resulting evaluation points are, by construction, in $\mathbb{F}_{2^{32}}$.
Apply Karatsuba (only once) to operate in $\mathbb{F}_{2^{16}}$ and have the results fit in the 32-bit CPU registers.

Multiplications

Butterfly twiggle factors

Multiplication step: $\beta \cdot p_1(s_1(\beta))$, $\beta = (v + w) \in v_{l+m/2-i} + \mathbf{W}_{l-i}$

- Precompute and store all the β values
- Notice that $\gamma \cdot \beta = \gamma \cdot (v + w) = \gamma \cdot v + \gamma \cdot w$
  - $\gamma \cdot v$ can be computed with a binary circuit (v are fixed)
  - $\gamma \cdot w$ can be computed in a subfield to reduce the number of bits

Pointwise multiplications

The resulting evaluation points are, by construction, in $\mathbb{F}_{2^{32}}$.
Apply Karatsuba (only once) to operate in $\mathbb{F}_{2^{16}}$ and have the results fit in the 32-bit CPU registers.

Multiplications

Butterfly twiggle factors

Multiplication step: $\beta \cdot p_1(s_1(\beta))$, $\beta = (v + w) \in v_{l+m/2-i} + \mathbf{W}_{l-i}$

- Precompute and store all the β values
- Notice that $\gamma \cdot \beta = \gamma \cdot (v + w) = \gamma \cdot v + \gamma \cdot w$
  - $\gamma \cdot v$ can be computed with a binary circuit (v are fixed)
  - $\gamma \cdot w$ can be computed in a subfield to reduce the number of bits

Pointwise multiplications

The resulting evaluation points are, by construction, in $\mathbb{F}_{2^{32}}$.
Apply Karatsuba (only once) to operate in $\mathbb{F}_{2^{16}}$ and have the results fit in the 32-bit CPU registers.

Masking HQC

HQC operations are linear $\rightarrow$ boolean masking of the entire scheme

$$a \mapsto [\![a]\!] = (a_1, \cdots, a_d) \text{ such that } \bigoplus_{i=1}^{d} a_i = a$$

Our implementation

More shares $\rightarrow$ slower performance, higher security

We chose $d = 2$ shares:

$$[\![a]\!] = (a \oplus x, x)$$

- a is the secret value
- x is a random value (from the TRNG)

Masking HQC

HQC operations are linear $\rightarrow$ boolean masking of the entire scheme

$$a \mapsto [\![a]\!] = (a_1, \cdots, a_d) \text{ such that } \bigoplus_{i=1}^{d} a_i = a$$

Our implementation

More shares $\rightarrow$ slower performance, higher security

We chose $d = 2$ shares:

$$[\![a]\!] = (a \oplus x, x)$$

- a is the secret value
- x is a random value (from the TRNG)

Masking HQC

HQC operations are linear $\rightarrow$ boolean masking of the entire scheme

$$a \mapsto [\![a]\!] = (a_1, \cdots, a_d) \text{ such that } \bigoplus_{i=1}^{d} a_i = a$$

Our implementation

More shares $\rightarrow$ slower performance, higher security

We chose $d = 2$ shares:

$$[\![a]\!] = (a \oplus x, x)$$

- a is the secret value
- x is a random value (from the TRNG)

Keygen

Mask x and y, perform the same operations of the unmasked version

Function KEYGEN()

$h \xleftarrow{\$} \mathcal{R}$;

$(x, y) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;

$[\![y]\!] \leftarrow (y + \sum \mathtt{rnd}_i)$;

$[\![s]\!] \leftarrow h[\![y]\!]$;

$[\![s]\!] \leftarrow x + [\![s]\!]$;

$\mathcal{K}_{pri} \leftarrow (x, y)$;

$\mathcal{K}_{pub} \leftarrow (\textsc{Collapse}([\![h]\!]), s)$;

return $(\mathcal{K}_{pri}, \mathcal{K}_{pub})$

Encryption

Mask e, r_1 and r_2, perform the same operations of the unmasked version

Function ENCRYPT($\mathcal{K}_{pub}, m$)

$e \xleftarrow{\$} \mathcal{R}_{\omega_e}$;

$(r_1, r_2) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;

$[\![r_1]\!] \leftarrow (r_1 + \sum \mathtt{rnd}_i)$;

$[\![r_2]\!] \leftarrow (r_2 + \sum \mathtt{rnd}_i)$;

$[\![u]\!] \leftarrow [\![r_1]\!] + h[\![r_2]\!]$;

$[\![m]\!] \leftarrow (m + \sum \mathtt{rnd}_i)$;

$[\![v]\!] \leftarrow [\![m]\!]\mathbf{G} + s[\![r_2]\!] + e$;

return (COLLAPSE($[\![u]\!]$), COLLAPSE($[\![v]\!]$))

Decryption

Function DECRYPT($\mathcal{K}_{pri}, c$)

$[\![y]\!] \leftarrow (y + \sum \mathtt{rnd}_i)$;
$[\![uy]\!] \leftarrow u[\![y]\!]$;
$[\![\bar{m}]\!] \xleftarrow{\$} (\{0,1\}^k)^d$;
$[\![\bar{c}]\!] \leftarrow [\![\bar{m}]\!]\mathbf{G}$;
$a \leftarrow$ COLLAPSE$((v\ , [\![\bar{c}]\!]))$;
$m \leftarrow$
DECODE(COLLAPSE$(a, [\![uy]\!])$);
return COLLAPSE$(m\ , [\![\bar{m}]\!])$;

Generate a random message $[\![\bar{m}]\!]$
Encode it $[\![\bar{c}]\!] = [\![\bar{m}]\!]\mathbf{G}$
Add the first half of the ciphertext
Collapse it together with the second half of the ciphertext
Decode the result
Remove the random message

Linearity of the RMRS code

$$\text{ENCODE}(m_1 + m_2) = \text{ENCODE}(m_1) + \text{ENCODE}(m_2)$$

Decryption

Function DECRYPT($\mathcal{K}_{pri}, c$)

$[\![y]\!] \leftarrow (y + \sum \mathtt{rnd}_i)$;
$[\![uy]\!] \leftarrow u[\![y]\!]$;
$[\![\bar{m}]\!] \xleftarrow{\$} (\{0,1\}^k)^d$;
$[\![\bar{c}]\!] \leftarrow [\![\bar{m}]\!]\mathbf{G}$;
$a \leftarrow$ COLLAPSE$((v\ , [\![\bar{c}]\!]))$;
$m \leftarrow$
DECODE(COLLAPSE$(a, [\![uy]\!])$);
return COLLAPSE$(m\ , [\![\bar{m}]\!])$;

Generate a random message $[\![\bar{m}]\!]$
Encode it $[\![\bar{c}]\!] = [\![\bar{m}]\!]\mathbf{G}$
Add the first half of the ciphertext
Collapse it together with the second half of the ciphertext
Decode the result
Remove the random message

Linearity of the RMRS code

$$\text{ENCODE}(m_1 + m_2) = \text{ENCODE}(m_1) + \text{ENCODE}(m_2)$$

Experimental results

Measuring time

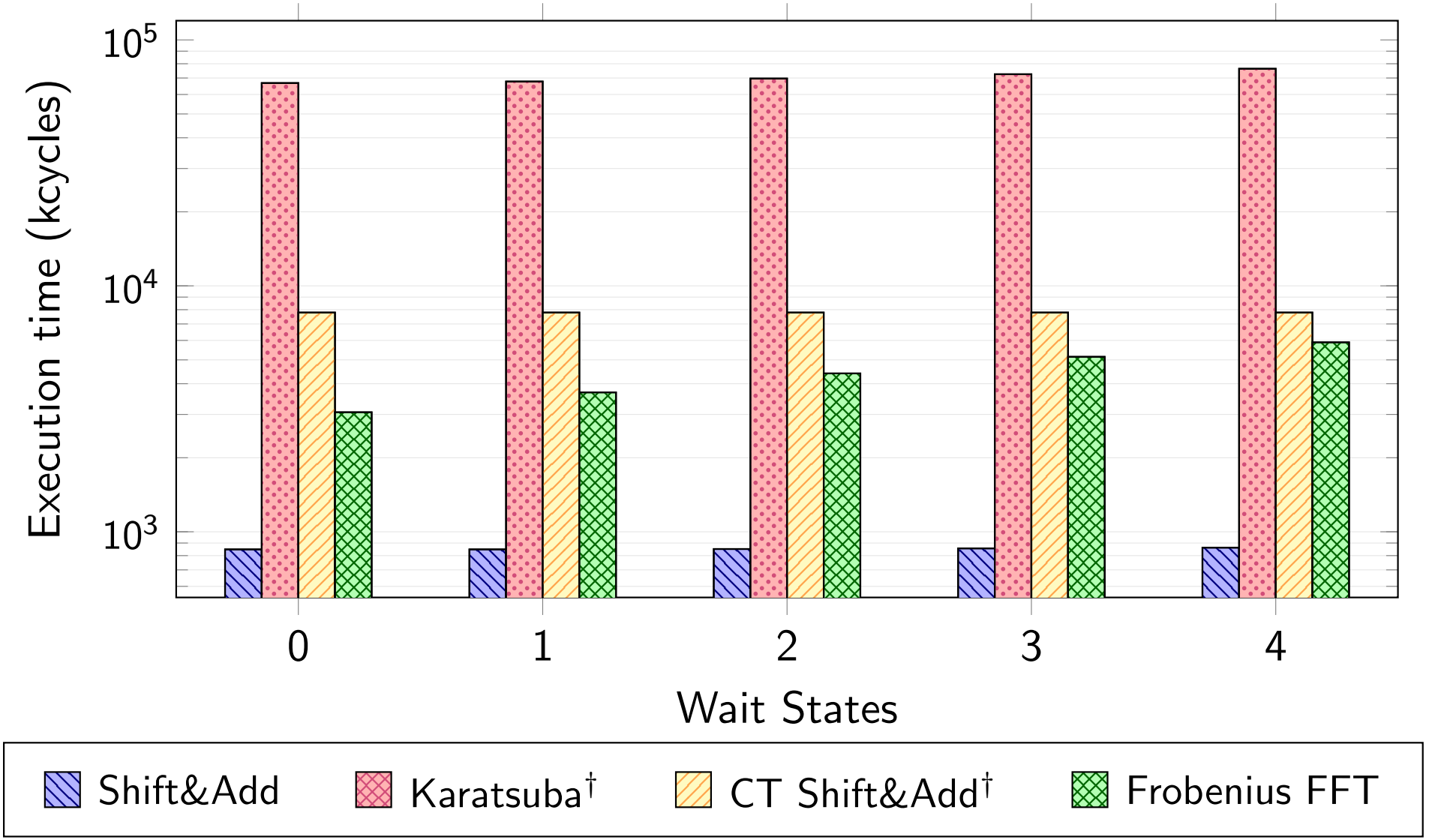
Performance evaluation metric: amount of clock cycles

- `DWT_CYCCNT` register for Cortex-M4
- `rtdsc` instruction for x86

Measurements are averaged over 1000 runs.
Repeat measurements while varying the amount of memory wait states.

- 16, 32, 48, 64 and 80 MHz on ARM
- 2GHz + no TurboBoost on x86

Timing results



Test Vector Leakage Assessment

Statistical t-test

Determine how much the means of two populations differ by evaluating

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}}$$

A t-value $|t| < 4.5$ implies that the populations have the same mean with 99.999% confidence.

Populations to be measured (1000 rounds each)

- fixed inputs
- random inputs

Randomness is always generated by the onboard TRNG.

Constant-time: multiplication

Table: t-values for the multiplication algorithms

	Shift&Add	**Karatsuba**	**CT Shift&Add**	**Frobenius FFT**
`ARM`	55.748	0.000	0.000	0.000
`x86`	20.936	3.644	3.371	1.232

All algorithms except plain S&A are constant-time, we choose

- Frobenius FFT on ARM
- Karatsuba (with AVX) on x86

Constant-time: PKE

Round	Version	Mul	CT?	Encrypt x86	Encrypt ARM	Decrypt x86	Decrypt ARM
3	`reference`	Shift&Add	○	10.65	29.19	5.59	39.23
	`sparsemul`	CT Shift&Add†	•	9.38	23.12	7.59	49.56
4	`reference`	Karatsuba†	•	1.27	1.04	0.97	0.30
	`sparsemul`	CT Shift&Add†	•	0.29	0.07	1.51	1.64
	`fastmul`	Frobenius FFT	•	3.36	0.56	1.70	1.99

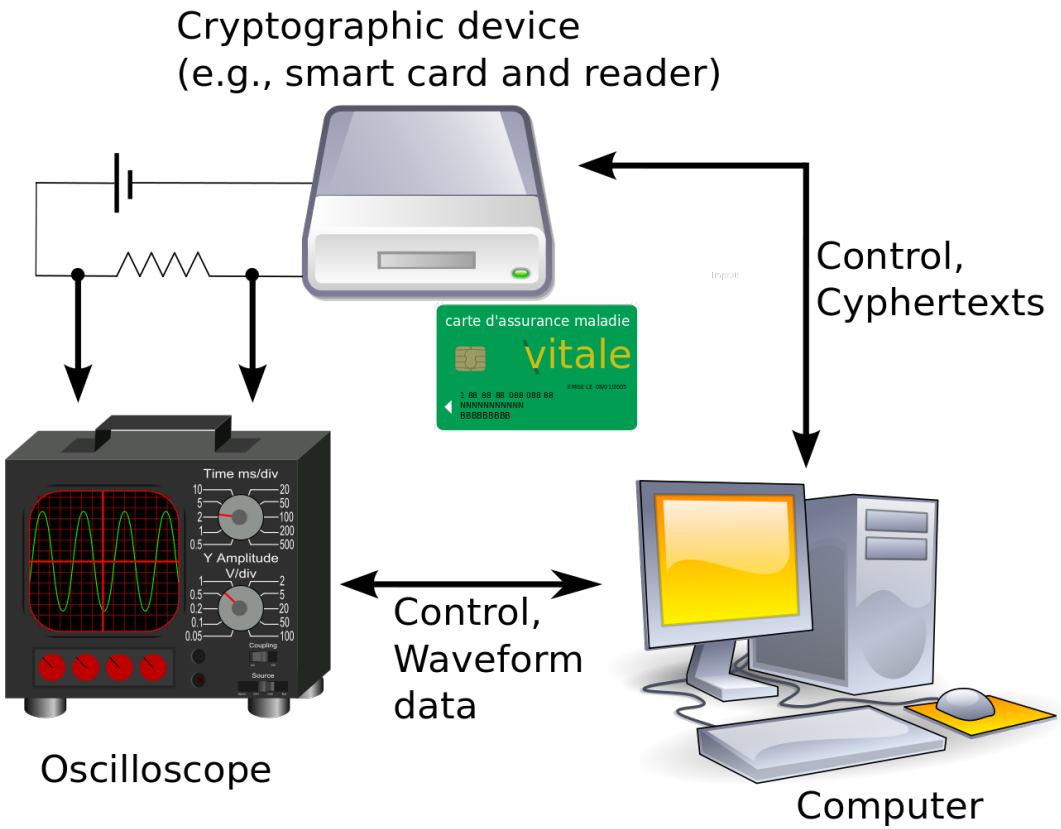
Constant-time: KEM

Round	Version	Mul	CT?	Encaps x86	Encaps ARM	Decaps x86	Decaps ARM
3	`reference`	Shift&Add	○	58.42	28.65	20.71	18.44
	`sparsemul`	CT Shift&Add†	•	6.75	14.29	5.81	23.70
4	`reference`	Karatsuba†	•	3.77	4.01	4.56	0.26
	`sparsemul`	CT Shift&Add†	•	1.58	2.17	1.04	0.22
	`fastmul`	Frobenius FFT	•	4.31	4.25	2.41	0.53

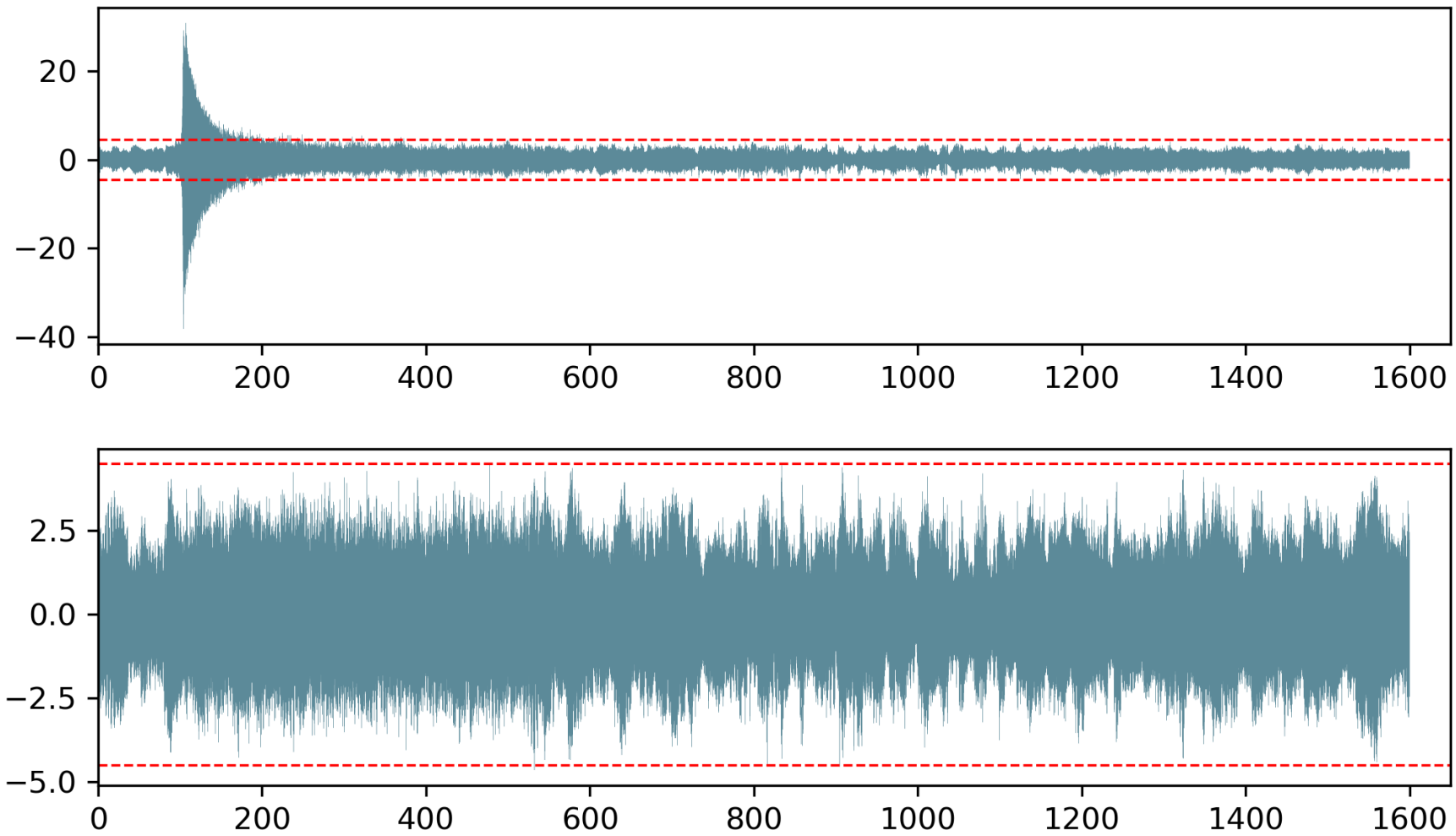
Power measurements

Setup

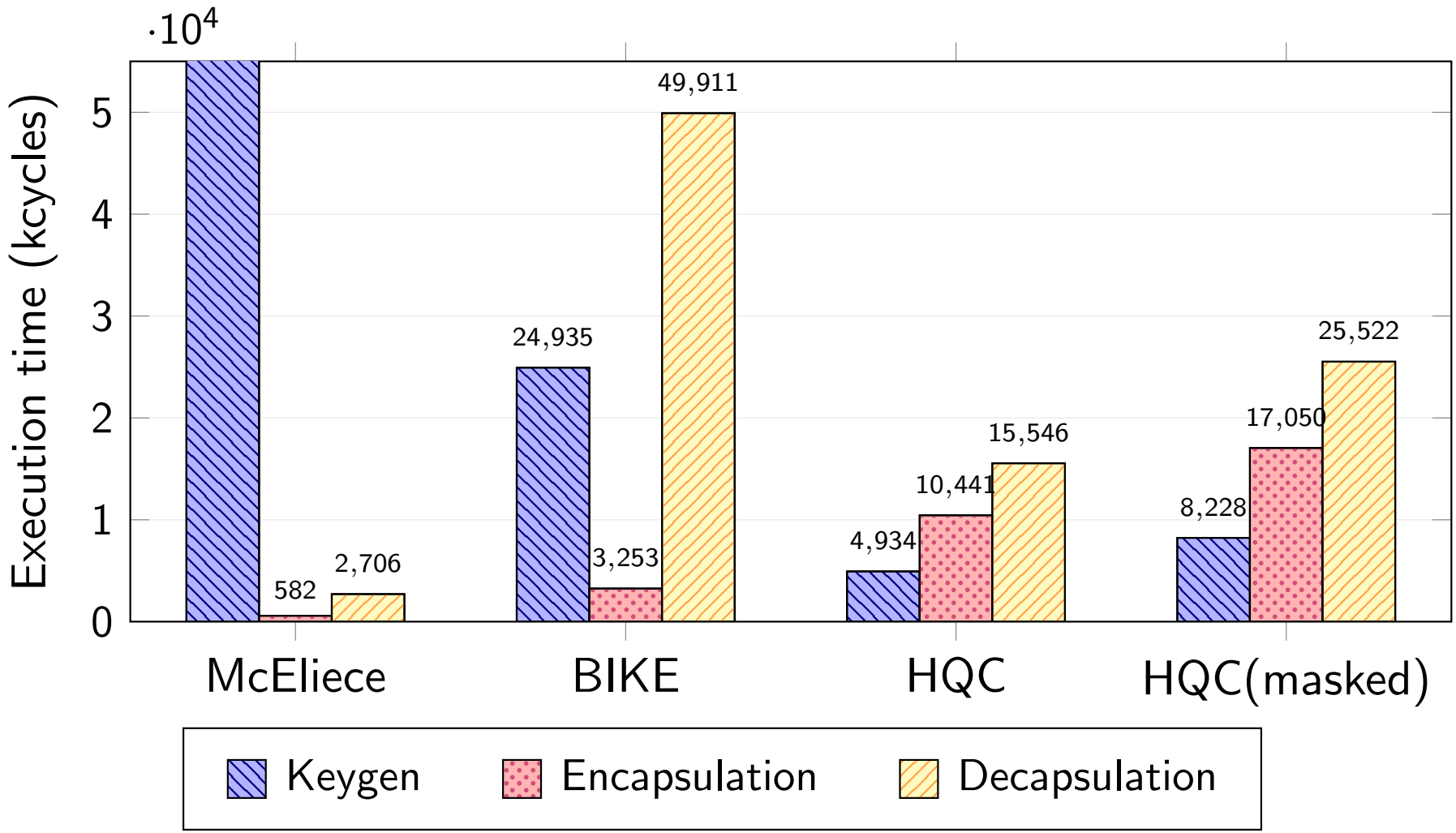
Measure the EM radiation emitted by the board package (non invasive)



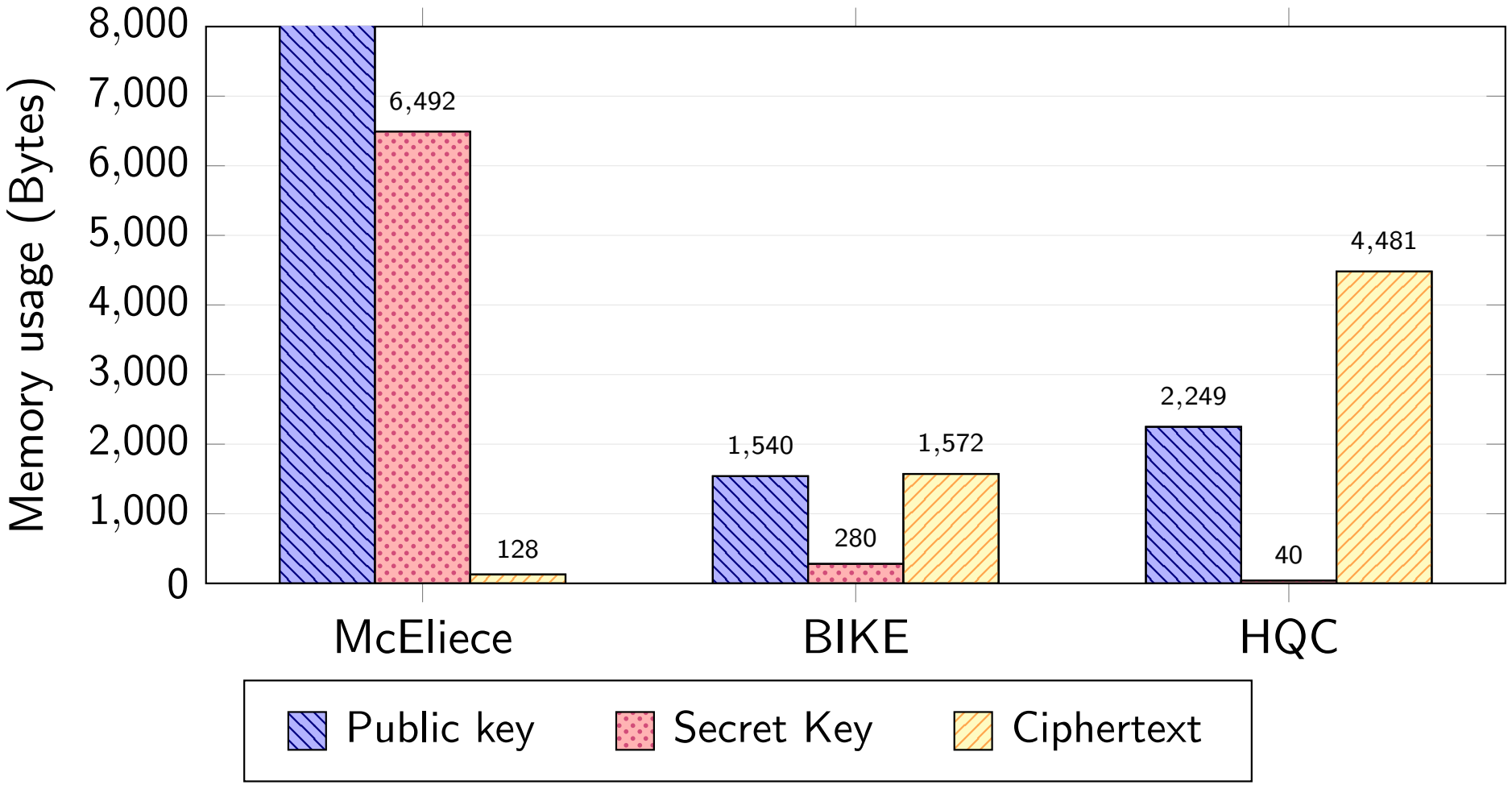
EM t-values: decoder



Execution time



Memory


Memory usage (Bytes)
8,000
7,000
6,000
5,000
4,000
3,000
2,000
1,000
0
6,492
128
1,540
280
1,572
2,249
40
4,481
McEliece
BIKE
HQC
Public key
Secret Key
Ciphertext

Conclusions

Results

Our implementation

- Fast and secure multiplication algorithms
- Constant-time PKE and KEM
- Efficient masking scheme for the PKE

Future works

Extension of the masking scheme to the KEM

Thanks for your attention!

Questions?

Appendix

Novelpoly basis

Cantor basis

Vector spaces and bases

$\mathbb{F}_2^m$: vector space of m-bit strings over $\mathbb{F}_2$, with m power of 2
Elements of $\mathbb{F}_2^m$ are *vectors*, linear combinations of the *basis vectors*
Basis vectors: $\mathbf{v} = (v_1, \cdots, v_m)$

Canonical basis

$$\mathbf{v} = (1, \alpha, \alpha^2, \cdots, \alpha^{m-1})$$

Cantor basis

$$\mathbf{v} = (v_1, \cdots, v_m)$$

$$v_1 = 1 \qquad v_i^2 + v_i = v_{i-1}$$

Subspaces and vanishing polynomials

$$v_1 = 1 \qquad v_i^2 + v_i = v_{i-1}$$

Subspaces induced by Cantor basis

$$\mathbf{W}_0 := \{0\}, \qquad \mathbf{W}_i := span\{v_0, \cdots, v_{i-1}\}$$

with $\mathbf{W}_0 \subset \mathbf{W}_1 \subset \cdots \subset \mathbf{W}_m = \mathbb{F}_2^m$ and $\dim(\mathbf{W}_i) = i$.

Vanishing polynomials

$$s_i(x) = \prod_{a \in \mathbf{W}_i} (x - a)$$

with $\deg(s_i(x)) = |\mathbf{W}_i| = 2^i$.

Novelpoly basis

Properties of the vanishing polynomials

- linearity: $s_i(x) + s_i(y) = s_i(x + y)$
- recursivity: $s_{i+j}(x) = s_i(s_j(x))$, $s_i(v_{i+j}) = v_j$
- $s_i(v_i) = 1$ for v_i element of the Cantor basis ([CCK+17])
- all v_i are roots of $s_{i+1}(x)$

Novelpoly basis

Cantor basis $\rightarrow$ Subspaces $\rightarrow$ Vanishing polynomials $\rightarrow$ Novelpoly basis

$$X_k(x) = \prod (s_i(x))^{b_i} \qquad \text{with } k = \sum_{i=0}^{\log_2 n} b_i 2^i$$

with $\deg(X_k(x)) = k$.

Fujisaki-Okamoto transformation

Key Encapsulation Mechanism

Let $\mathcal{G}, \mathcal{H}, \mathcal{K}$ be hash functions and $\mathcal{E}$ an instance of HQC.PKE

Encapsulation

- Generate the seed $\mathbf{m} \xleftarrow{\$} \mathbb{F}_2^k$ and the randomness $\theta \leftarrow \mathcal{G}(\mathbf{m})$
- Generate the ciphertext $\mathbf{c} \leftarrow (\mathbf{u}, \mathbf{v}) = \mathcal{E}.\text{Encrypt}(\text{pk}, \mathbf{m}, \theta)$
- Derive the symmetric key $K = \mathcal{K}(\mathbf{m}, \mathbf{c})$
- Calculate $\mathbf{d} \leftarrow \mathcal{H}(\mathbf{m})$
- Send the pair $(\mathbf{c}, \mathbf{d})$

Key Encapsulation Mechanism

Let $\mathcal{G}, \mathcal{H}, \mathcal{K}$ be hash functions and $\mathcal{E}$ an instance of HQC.PKE

Decapsulation

- Decrypt $\mathbf{m}' \leftarrow \mathcal{E}.\text{Decrypt}(\text{sk}, \mathbf{c})$
- Compute $\theta' \leftarrow \mathcal{G}(\mathbf{m}')$ and $\mathbf{c}' \leftarrow \mathcal{E}.\text{Encrypt}(\text{pk}, \mathbf{m}', \theta')$
- Check that $\mathbf{c} = \mathbf{c}'$ and $\mathbf{d} = \mathcal{H}(\mathbf{m}')$
- Derive the shared key $K \leftarrow \mathcal{K}(\mathbf{m}, \mathbf{c})$

References I


Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillipe Gaborit, Shay Gueron, Tim Guneysu, Carlos Aguilar Melchor, Rafael Misoczki, Edoardo Persichetti, Nicolas Sendrier, Jean-Pierre Tillich, Gilles Zemor, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann, *BIKE Supporting Documentation*, `https://bikesuite.org`, 2022.


Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang, *Classic McEliece Supporting Documentation*, `https://classic.mceliece.org`, 2022.

References II

 Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg, *On the inherent intractability of certain coding problems (corresp.)*, IEEE Trans. Inf. Theory **24** (1978), no. 3, 384–386.

 Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang, *Faster multiplication for long binary polynomials*, CoRR **abs/1708.09746** (2017).

 Ming-Shing Chen, Tung Chou, and Markus Krausz, *Optimizing BIKE for the intel haswell and ARM cortex-m4*, IACR Trans. Cryptogr. Hardw. Embed. Syst. **2021** (2021), no. 3, 97–124.

References III


Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder, *Don't reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE*, IACR Trans. Cryptogr. Hardw. Embed. Syst. **2022** (2022), no. 3, 223–263.


Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han, *Novel polynomial basis and its application to reed-solomon erasure codes*, 55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014, IEEE Computer Society, 2014, pp. 316–325.

References IV


Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges, *Hamming Quasi-Cyclic (HQC)*, `https://pqc-hqc.org/`, 2018.


Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl, *A Power Side-Channel Attack on the Reed-Muller Reed-Solomon Version of the HQC Cryptosystem*, Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings (Jung Hee Cheon and Thomas Johansson, eds.), Lecture Notes in Computer Science, vol. 13512, Springer, 2022, pp. 327–352.

References V


Thomas Schamberger, Julian Renner, Georg Sigl, and Antonia Wachter-Zeh, *A Power Side-Channel Attack on the CCA2-Secure HQC KEM*, Smart Card Research and Advanced Applications - 19th International Conference, CARDIS 2020, Virtual Event, November 18-19, 2020, Revised Selected Papers (Pierre-Yvan Liardet and Nele Mentens, eds.), Lecture Notes in Computer Science, vol. 12609, Springer, 2020, pp. 119–134.


Guillaume Wafo-Tapa, Slim Bettaieb, Loïc Bidoux, Philippe Gaborit, and Etienne Marcatel, *A practicable timing attack against HQC and its countermeasure*, Adv. Math. Commun. **16** (2022), no. 3, 621–642.